Claude Code DSL Orchestration: Complete Reference

Domain-Specific Language for Agent Orchestration Version: 2.0.0 **Date**: 2025-10-19 **Status**: Comprehensive Guide ---

Table of Contents

Part I: Foundations

1. Introduction

The Claude Code DSL (Domain-Specific Language) provides a mathematical framework for composing agents, skills, and commands into powerful orchestration workflows.

1.1 Design Philosophy

```
┌─────────────────────────────────────────────────────┐
│  "Make simple things simple, complex things possible"  │
└─────────────────────────────────────────────────────┘


Level 1-2: Intuitive for beginners
Level 3-4: Powerful for practitioners
Level 5-6: Expressive for experts
```

1.2 Core Abstractions

Agents: Autonomous workers with specialized capabilities **Skills**: Knowledge modules that augment agents **Commands**: Operations and utilities **Workflows**: Composed orchestrations with execution semantics

1.3 When to Use This DSL

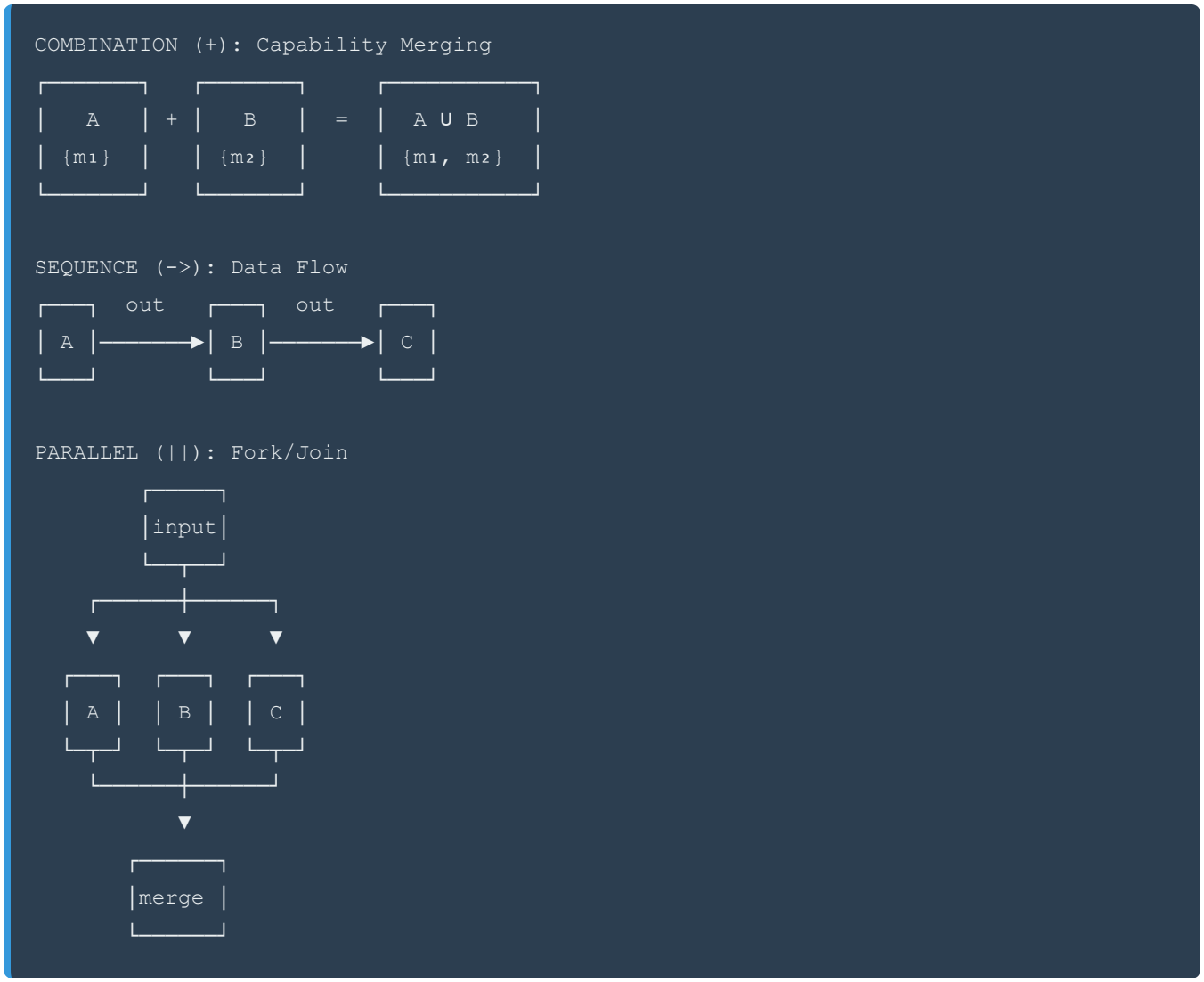
- ✅ Orchestrating multiple agents in parallel or sequence - ✅ Building complex multi-step workflows - ✅ Optimizing execution time and token budgets - ✅ Creating reusable workflow templates - ✅ Expressing complex dependencies as DAGs - ❌ Simple single-agent tasks (use direct invocation) - ❌ Tasks requiring imperative control flow (use scripts) ---

2. Core Operators

2.1 Operator Summary

| Operator | Name | Arity | Description | Example | |----------|------|-------|-------------|---------| | `+` | Combination | Binary | Merge capabilities | `agent + skill` | | `->` | Sequence | Binary | Pipeline execution | `a -> b` | | `\|\|` | Parallel | N-ary | Concurrent execution | `a \|\| b \|\| c` | | `:` | Specification | Binary | Task binding | `workflow : task` | | `=` | Assignment | Binary | Parameter binding | `cmd = param` | | `()` | Grouping | N-ary | Precedence control | `(a \|\| b) -> c` | | `[]` | Annotation | Unary | Metadata/constraints | `agent[budget=50K]` |

2.2 Operator Semantics



---

3. Mathematical Foundations

3.1 Type System

```
-- Base types
type Agent = Context -> Input -> (Output, Context)
type Skill = { methods :: [Method], properties :: [Property] }
type Command = forall a b. a -> b
type Workflow a b = a -> Result b

-- Composite types
data Composition
  = Sequential Agent Agent
  | Parallel [Agent]
  | Combined Agent Skill
  | Conditional Predicate Workflow Workflow

-- Type constraints
class Composable a b where
  compose :: a -> b -> c

instance Composable Agent Agent where
  compose a1 a2 = \ctx input ->
    let (out1, ctx1) = a1 ctx input
        (out2, ctx2) = a2 ctx1 out1
    in (out2, ctx2)
```

3.2 Category Theory Foundations

```
Category WORKFLOW where
  Objects: Agent, Skill, Result
  Morphisms: Workflow<A, B> :: A -> B

  Identity:
    id :: A -> A
    id(x) = x

  Composition:
    (∘) :: (B -> C) -> (A -> B) -> (A -> C)
    (f ∘ g)(x) = f(g(x))

  Laws:
    Left Identity:  id ∘ f = f
    Right Identity: f ∘ id = f
    Associativity:  (f ∘ g) ∘ h = f ∘ (g ∘ h)

Visual:
    A ─────f─────▶ B ─────g─────▶ C
    │                            ▲
    └────────────g ∘ f────────────┘
```

3.3 Algebraic Properties

```
COMBINATION (+):
  Monoid structure:
    Identity:       A + ∅ = A
    Associativity:  (A + B) + C = A + (B + C)
    Commutativity:  A + B = B + A

SEQUENCE (->):
  Category structure:
    Identity:       id -> A = A = A -> id
    Associativity:  (A -> B) -> C = A -> (B -> C)
    NOT Commutative: A -> B ≠ B -> A

PARALLEL (||):
  Symmetric monoidal structure:
    Identity:       A || ∅ = A
    Associativity:  (A || B) || C = A || (B || C)
    Commutativity:  A || B = B || A
    Braiding:       swap(A || B) = B || A
```

Part II: Complexity Levels

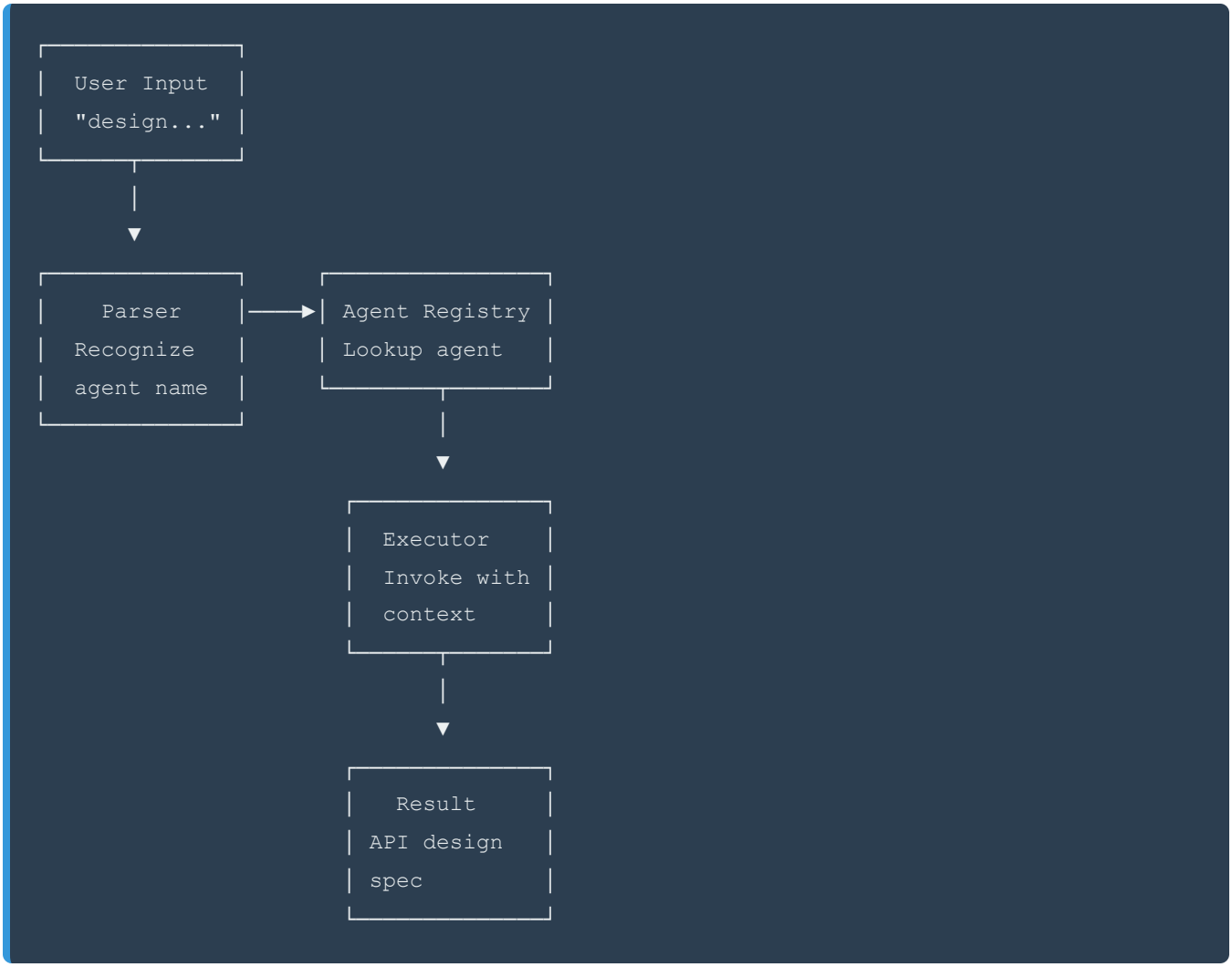
4. Level 1: Basic Invocation

Complexity: Minimal **Execution Model**: Single function call **Token Budget**: 5,000-15,000 **Time Estimate**: 2-5 minutes

4.1 Single Agent Invocation

```
api-architect : "design REST API for user authentication"
```

Execution Flow:



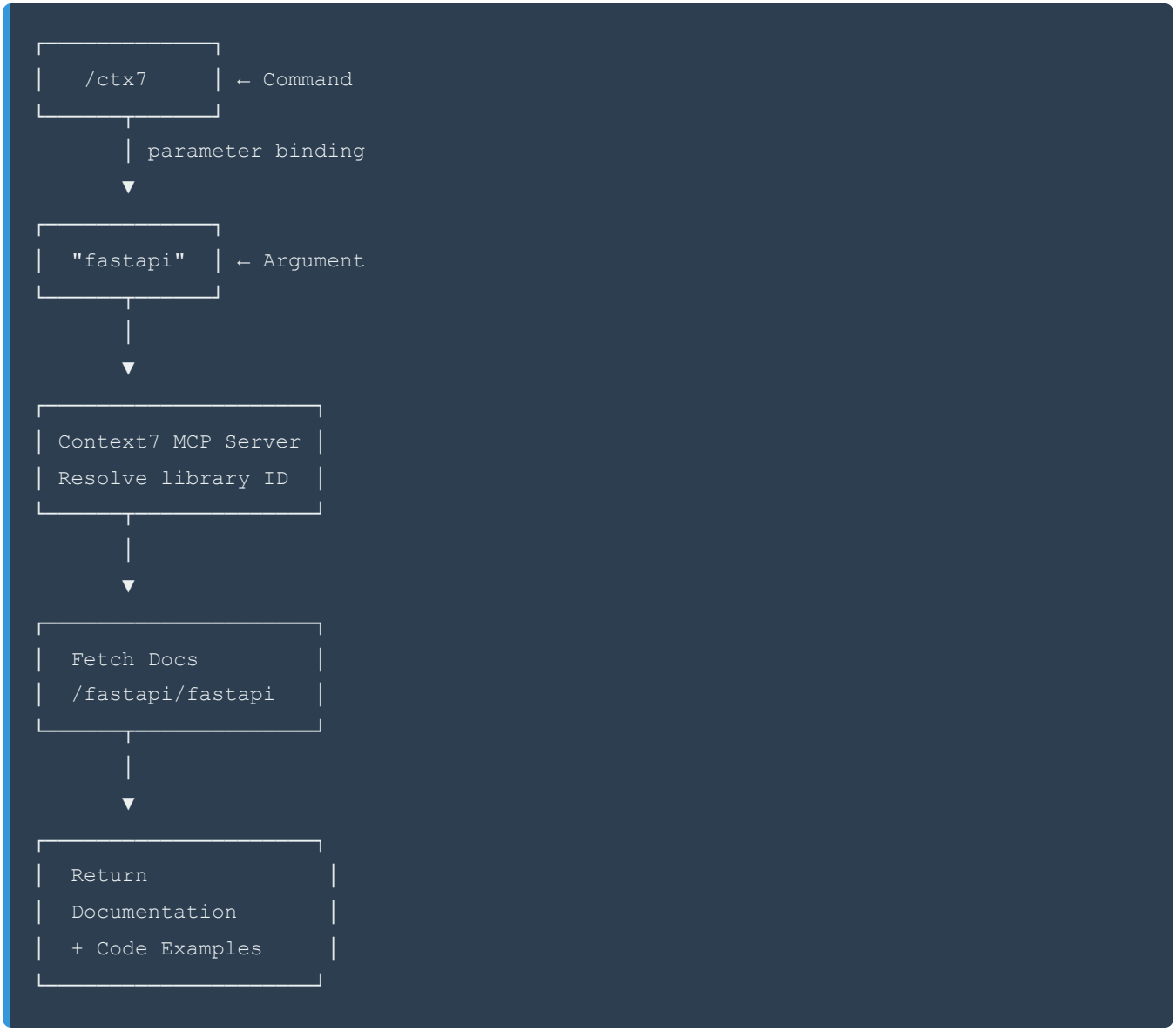
Mathematical Model:

```
f: Input -> Output

f("design REST API") = {
  openapi_spec,
  endpoint_definitions,
  authentication_flow
}

Simple function application, no composition
```

4.2 Single Command Execution

```
/ctx7 = fastapi
```

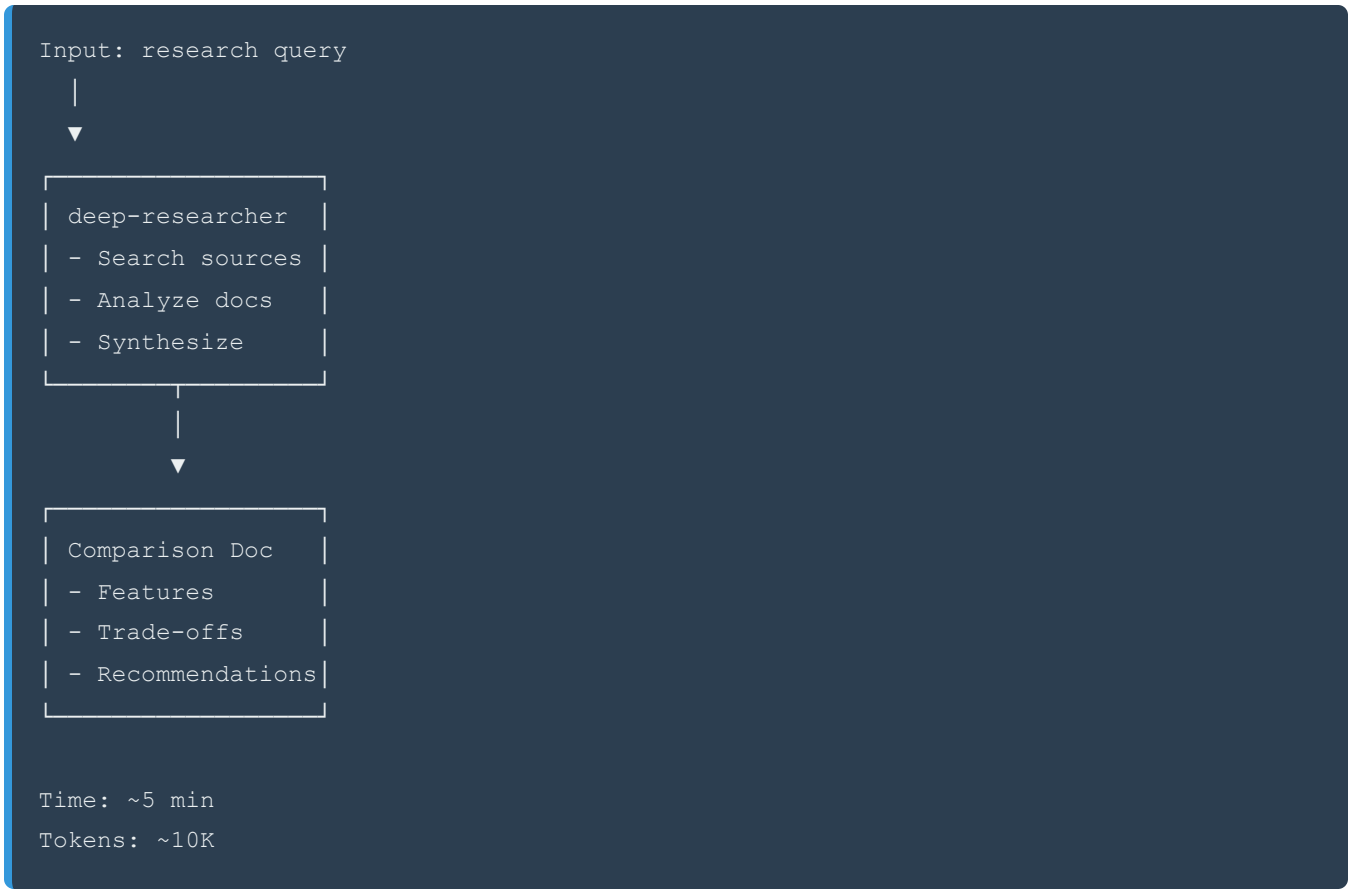
Execution Flow:



4.3 Level 1 Examples

Example 1.1: Quick Research

```
deep-researcher : "compare PostgreSQL vs MongoDB"
```



Example 1.2: Library Documentation

```
/ctx7 = express
```

```
/ctx7 command
  |
  ▼
Resolve "express" → /expressjs/express
  |
  ▼
Fetch comprehensive docs
  |
  ▼
docs/EXPRESS-CONTEXT7-REVIEW.md

Time: ~3 min
Tokens: ~5K
```

Example 1.3: Design Task

```
frontend-architect : "design dashboard with charts"
```

---

5. Level 2: Binary Operations

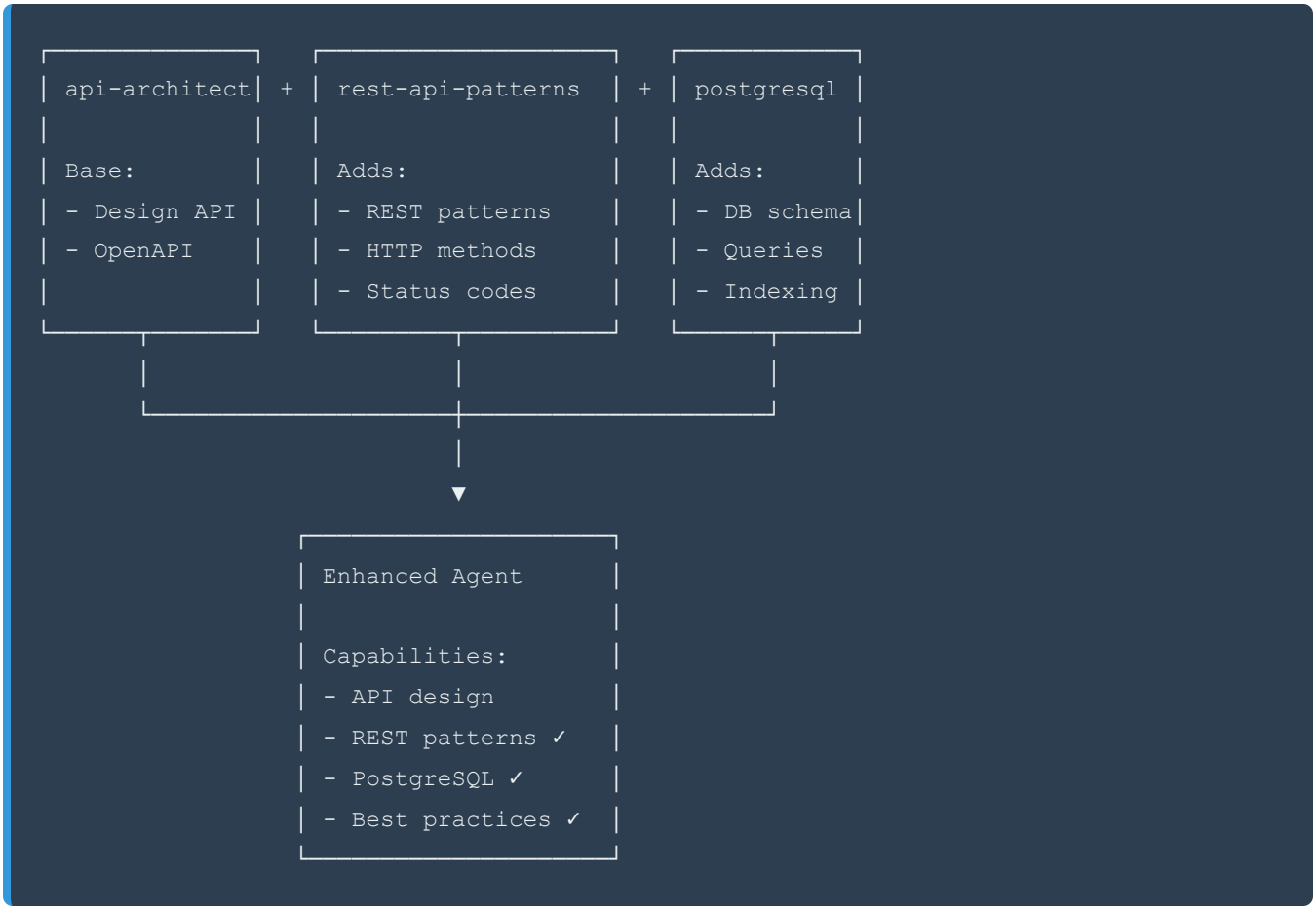
Complexity: Low **Execution Model**: Binary composition **Token Budget**: 10,000-30,000 **Time Estimate**: 5-15 minutes

5.1 Combination (+)

Semantics: Augment agent with additional capabilities

```
api-architect + rest-api-design-patterns + postgresql
```

Visual Model:



Mathematical Model:

```
Skill Algebra:
  S₁ ⊕ S₂ = { M₁ ∪ M₂, P₁ ∪ P₂, I₁ ∧ I₂ }

  Where:
    M = Methods
    P = Properties
    I = Invariants

Agent Augmentation:
  A + S = A' where capabilities(A') = capabilities(A) ∪ S

Properties:
  Associative:   (A + S₁) + S₂ = A + (S₁ + S₂)
  Commutative:   A + S₁ + S₂ = A + S₂ + S₁
  Identity:      A + ∅ = A
```

5.2 Sequence (->)

Semantics: Pipeline execution, output of first becomes input of second

```
research -> design -> implement
```

Visual Model:

```
Step 1: Research          Step 2: Design           Step 3: Implement
┌──────────────┐          ┌──────────────┐         ┌──────────────┐
│   research   │  output  │    design    │ output  │  implement   │
│              │─────────>│              │────────>│              │
│ Analyze reqs │  spec    │ Create API   │  code   │ Build system │
│ Find patterns│          │ Design DB    │  plan   │ Write tests  │
└──────────────┘          └──────────────┘         └──────────────┘
   T = 10 min               T = 15 min               T = 25 min

Total Time: 10 + 15 + 25 = 50 minutes (sequential)
Total Tokens: 15K + 20K + 30K = 65K tokens
```

Data Flow Diagram:

Mathematical Model:

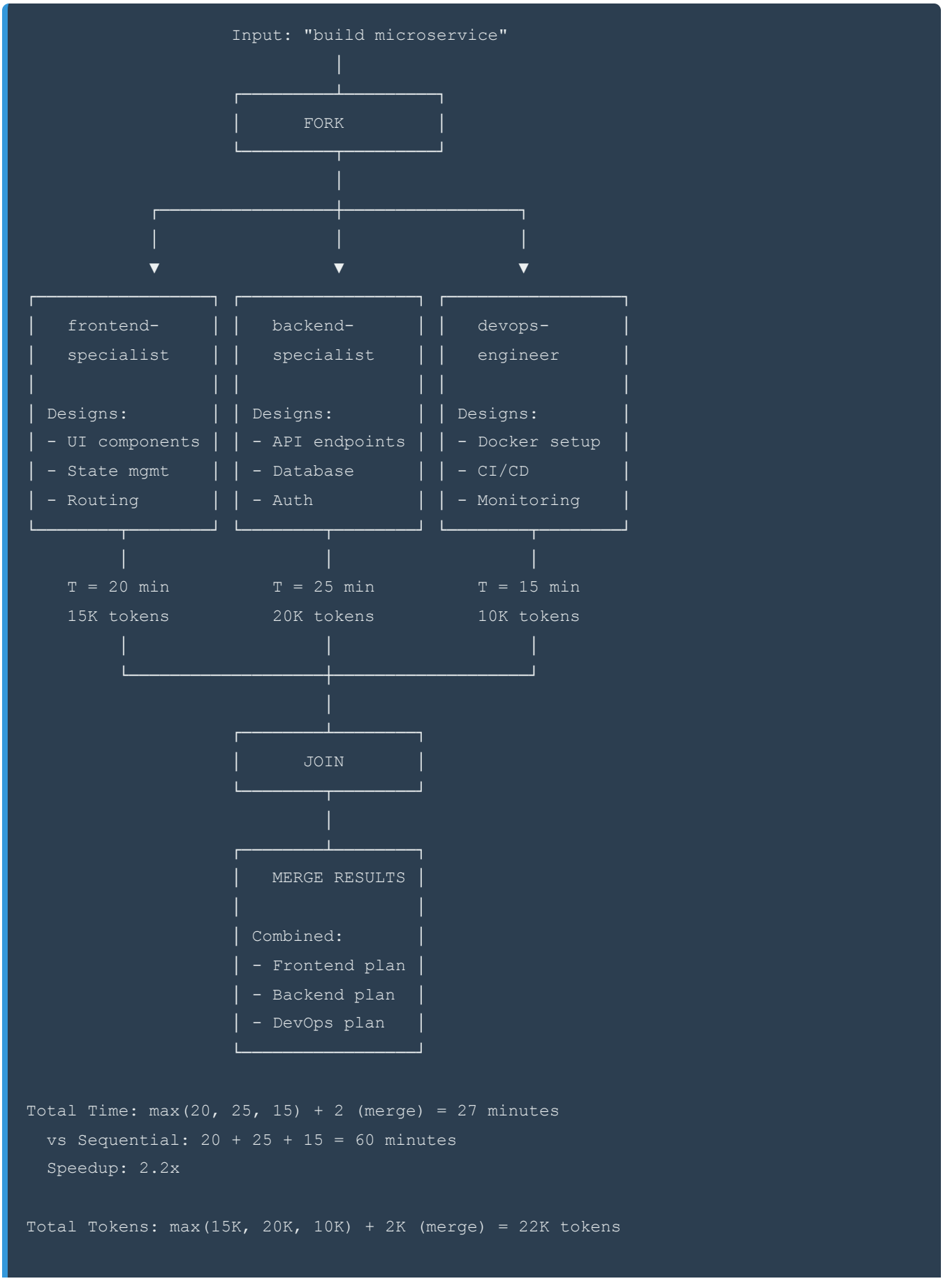
```
Function Composition:
  (f ∘ g)(x) = f(g(x))

In DSL:
  research -> design -> implement
  = implement ∘ design ∘ research
  = λx. implement(design(research(x)))

Properties:
  Associative:     (A -> B) -> C = A -> (B -> C)
  NOT Commutative: A -> B ≠ B -> A
  Identity:        id -> A = A -> id = A

Time Complexity:
  T(A -> B -> C) = T(A) + T(B) + T(C)
```

5.3 Parallel (||)

Semantics: Fork execution, run concurrently, join and merge results

```
frontend-specialist || backend-specialist || devops-engineer
```

Visual Model:




```
Total Time: max(20, 25, 15) + 2 (merge) = 27 minutes
  vs Sequential: 20 + 25 + 15 = 60 minutes
  Speedup: 2.2x

Total Tokens: max(15K, 20K, 10K) + 2K (merge) = 22K tokens
```

```
  vs Sequential: 45K tokens
  Savings: 51%
```

Mathematical Model:

```
Parallel Composition:
  (f ⊗ g)(x) = (f(x), g(x))

In DSL:
  A || B || C = λx. (A(x), B(x), C(x))

Properties:
  Associative:  (A || B) || C = A || (B || C)
  Commutative:  A || B = B || A
  Identity:     A || ∅ = A

Time Complexity:
  T(A || B || C) = max(T(A), T(B), T(C)) + T(merge)

Token Complexity:
  Tokens(A || B || C) ≈ max(Tokens(A), Tokens(B), Tokens(C)) + overhead
  overhead ≈ 5-10% of max
```

5.4 Level 2 Examples

Example 2.1: Enhanced API Design

```
api-architect + rest-api-design-patterns + postgresql + oauth2-authentication
```

```
api-architect (base)
    |
    ├─ + rest-api-design-patterns
    |    → Gains: RESTful conventions, HTTP semantics
    |
    ├─ + postgresql
    |    → Gains: Database schema design, SQL patterns
    |
    └─ + oauth2-authentication
         → Gains: Auth flows, JWT, security patterns

Result: API architect with comprehensive backend knowledge

Time: 20 min (single agent, augmented capabilities)
Tokens: 25K
```

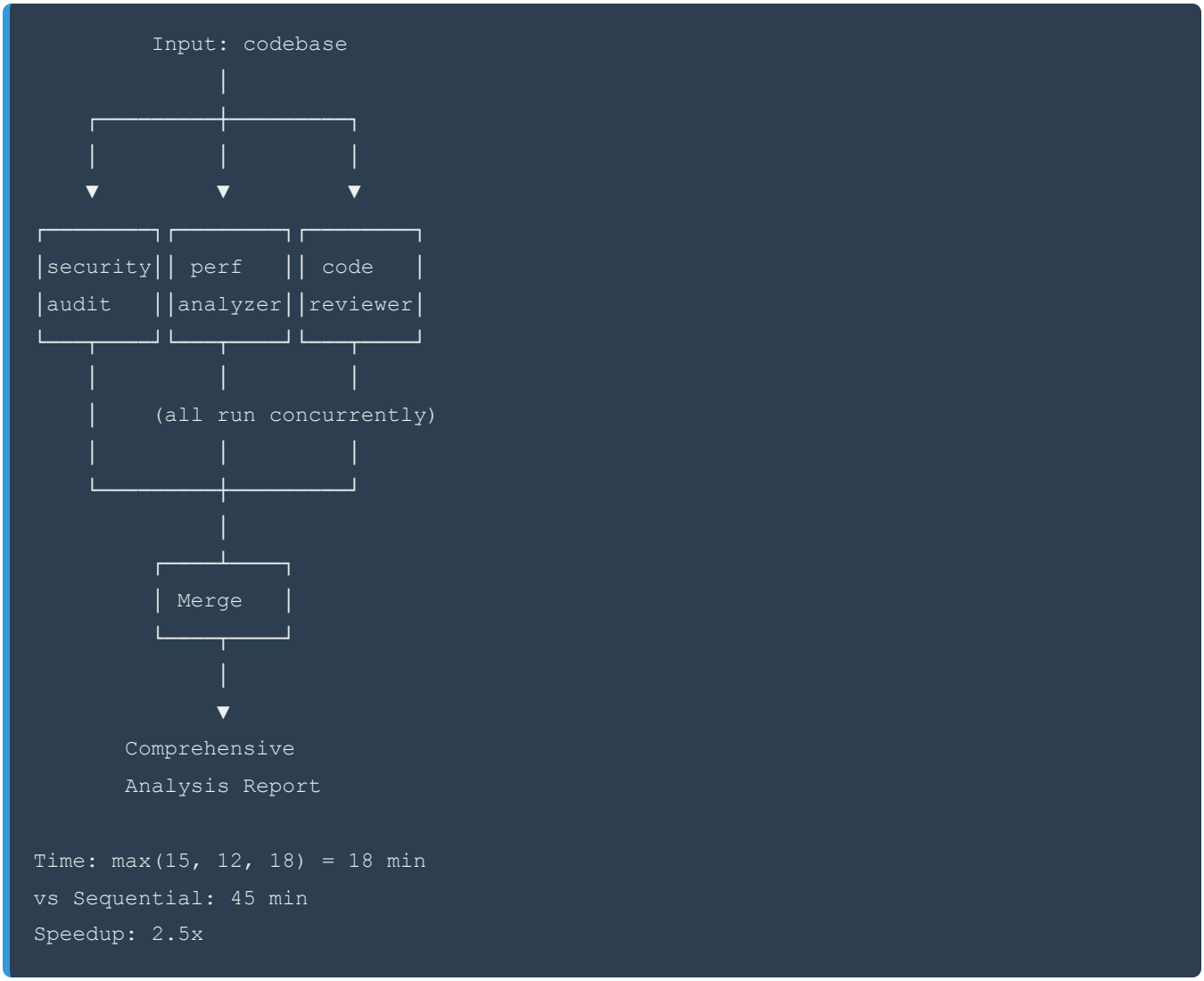
Example 2.2: Research Pipeline

```
deep-researcher -> api-architect -> docs-generator
```

```
Step 1: deep-researcher
  Input: "payment gateway integration"
  Output: Research document with:
    - Stripe API analysis
    - PayPal comparison
    - Security requirements
  Time: 15 min, 20K tokens

Step 2: api-architect
  Input: Research document
  Output: API design spec with:
    - Endpoint definitions
    - Payment flow diagrams
    - Error handling
  Time: 20 min, 25K tokens

Step 3: docs-generator
  Input: API design spec
  Output: Complete documentation:
    - API reference
    - Integration guide
    - Code examples
  Time: 15 min, 15K tokens

Total: 50 min, 60K tokens (sequential pipeline)
```

Example 2.3: Parallel Analysis

```
security-auditor || performance-analyzer || code-reviewer
```



---

6. Level 3: Parallel Streams

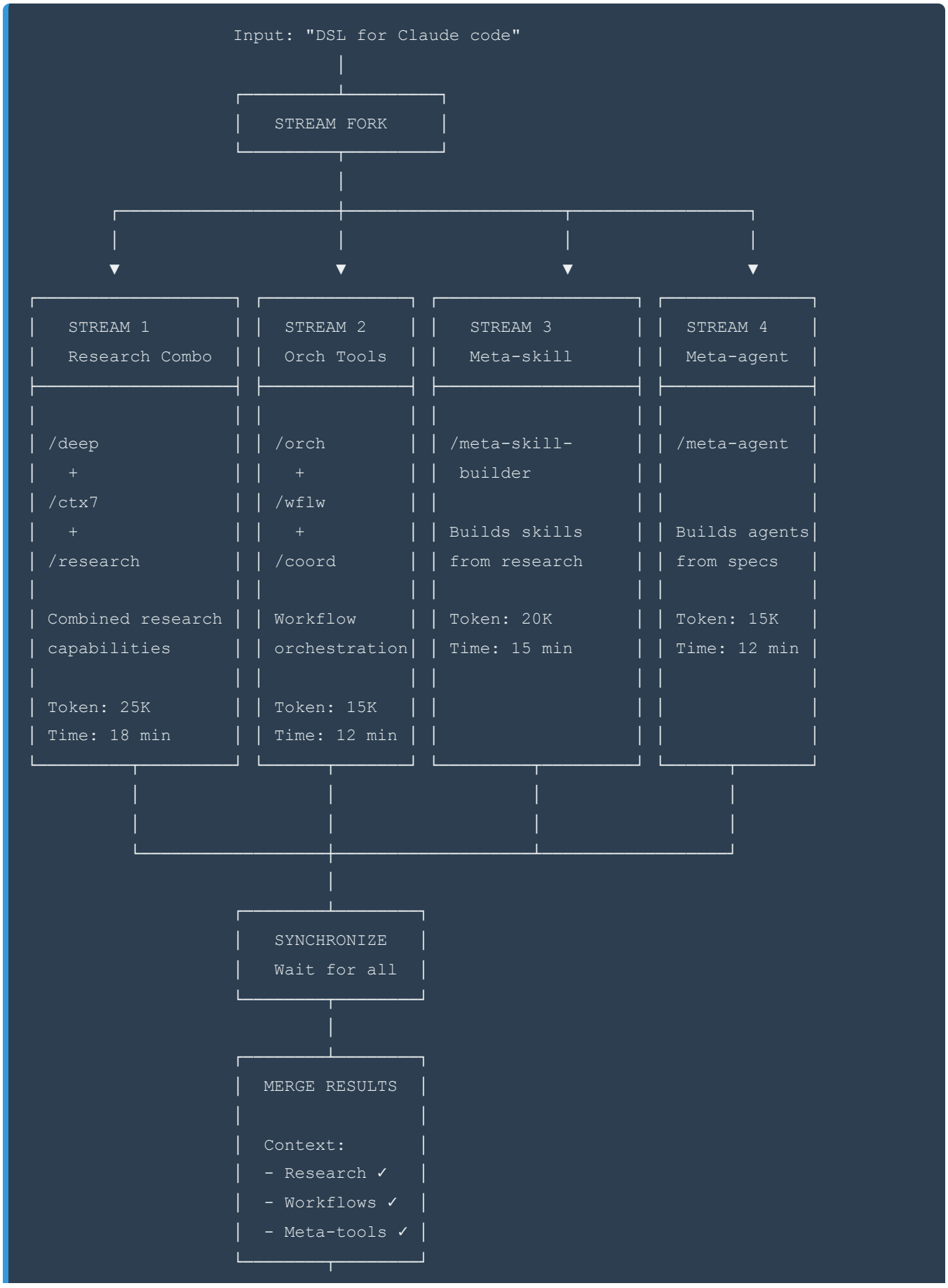
Complexity: Medium **Execution Model**: Multi-stream DAG execution **Token Budget**: 40,000-100,000 **Time Estimate**: 20-45 minutes

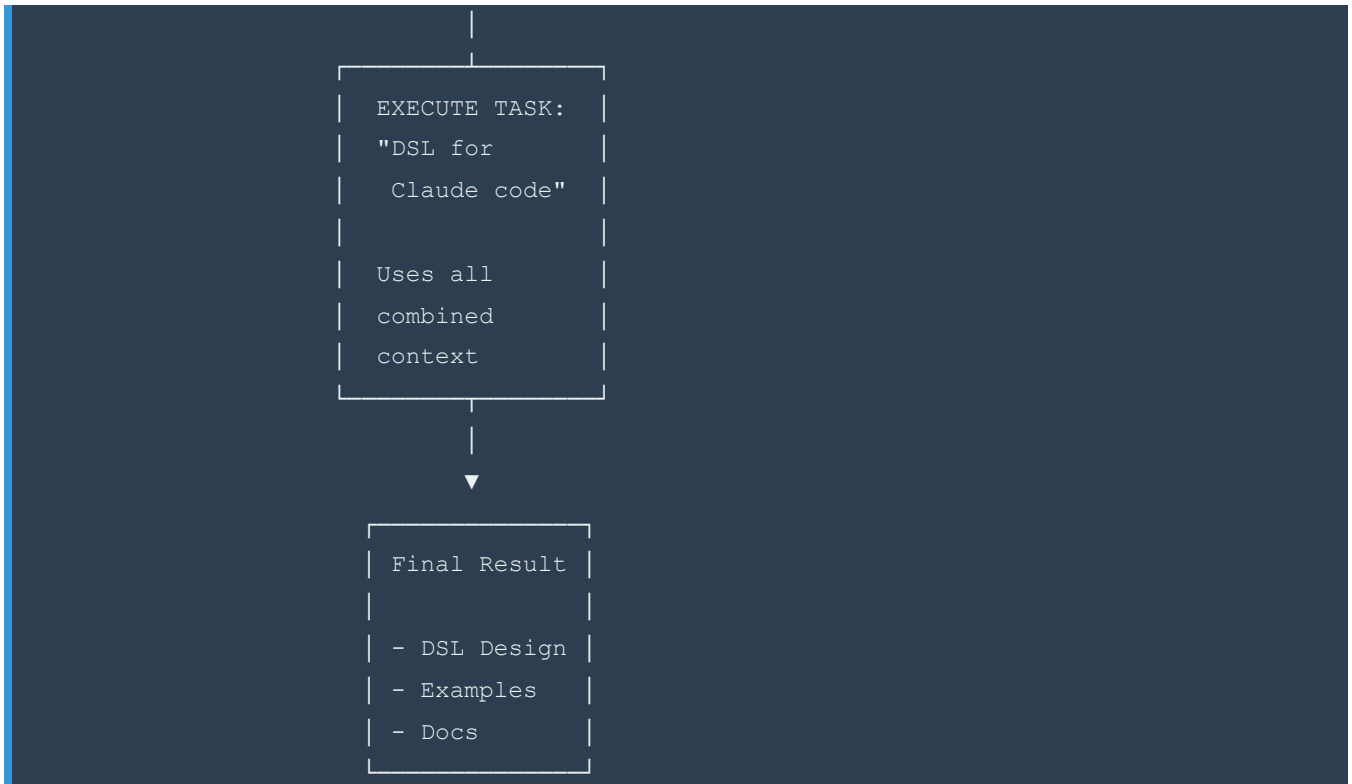
6.1 Multi-Stream Orchestration

Concept: Multiple independent execution streams running in parallel, each stream can be a combination or sequence.

```
(/deep + /ctx7 + /research || /orch /wflw /coord || /meta-skill-builder || /meta-agent)
: "DSL for Claude code"
```

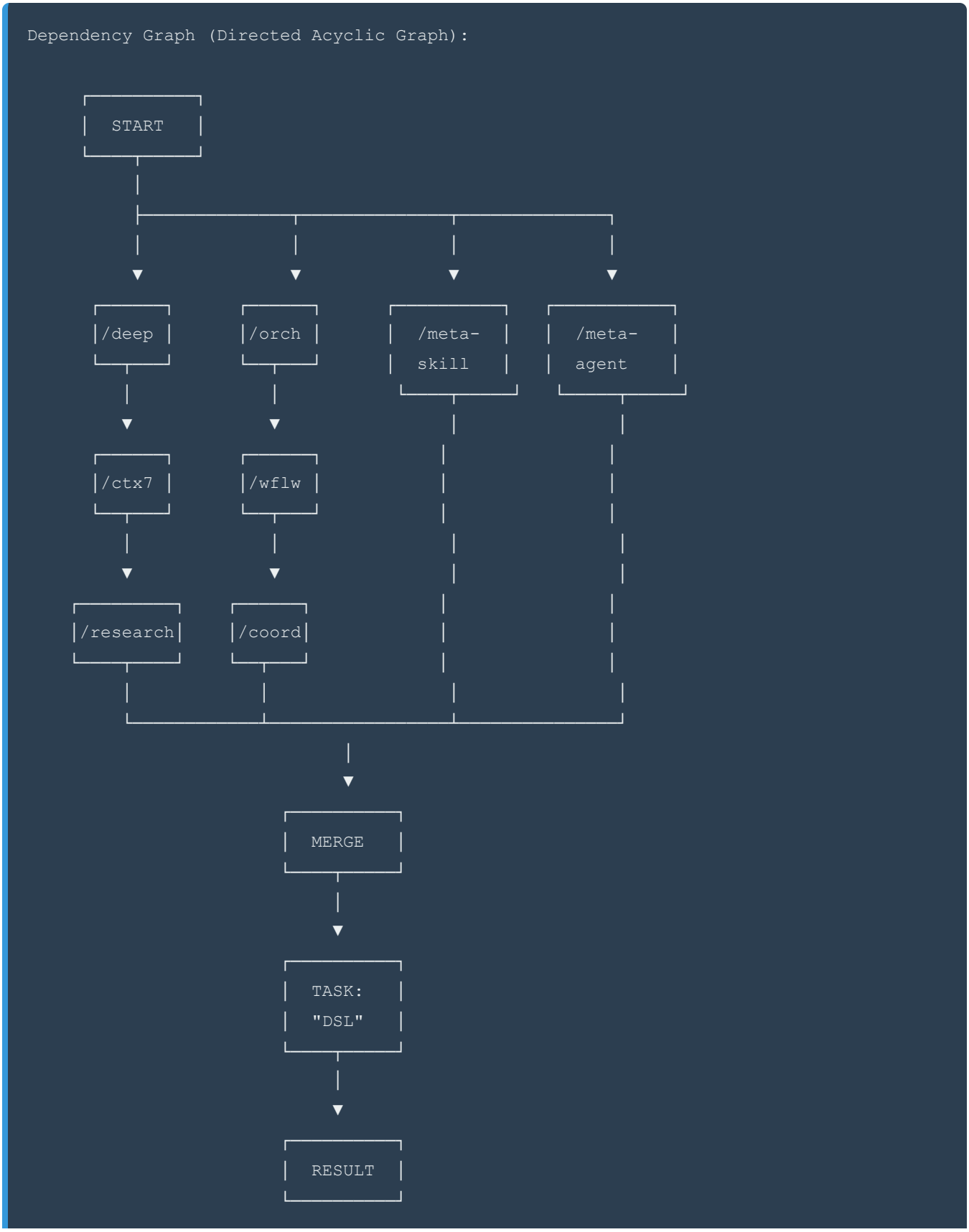
Visual Model:


Input: "DSL for Claude code"
STREAM FORK
STREAM 1
Research Combo
/deep
+
/ctx7
+
/research
Combined research capabilities
Token: 25K
Time: 18 min
STREAM 2
Orch Tools
/orch
+
/wflw
+
/coord
Workflow orchestration
Token: 15K
Time: 12 min
STREAM 3
Meta-skill
/meta-skill-builder
Builds skills from research
Token: 20K
Time: 15 min
STREAM 4
Meta-agent
/meta-agent
Builds agents from specs
Token: 15K
Time: 12 min
SYNCHRONIZE
Wait for all
MERGE RESULTS
Context:
- Research ✓
- Workflows ✓
- Meta-tools ✓




```
Total Time: max(18, 12, 15, 12) + 5 (merge) + 20 (task) = 45 min
  vs Sequential: 18 + 12 + 15 + 12 + 20 = 77 min
  Speedup: 1.7x

Total Tokens: max(25K, 15K, 20K, 15K) + 5K (merge) + 30K (task) = 60K
  vs Sequential: 105K tokens
  Savings: 43%
```

6.2 DAG Representation



```
Topological Sort: [START, /deep, /ctx7, /research, /orch, /wflw, /coord,
                   /meta-skill, /meta-agent, MERGE, TASK, RESULT]

Parallel Levels:
  Level 0: START
  Level 1: /deep, /orch, /meta-skill, /meta-agent (parallel)
  Level 2: /ctx7, /wflw (parallel, depend on level 1)
  Level 3: /research, /coord (parallel, depend on level 2)
  Level 4: MERGE (depends on all level 3)
  Level 5: TASK (depends on MERGE)
  Level 6: RESULT (depends on TASK)
```

6.3 Mathematical Model

```
-- Stream definition
data Stream a where
  Single   :: Agent -> Stream Agent
  Combined :: Agent -> Skill -> Stream (Agent, Skill)
  Sequence :: Stream a -> Stream b -> Stream (a, b)

-- Parallel execution
parallel :: [Stream a] -> Stream [a]
parallel streams = fork streams >>= join >>= merge

-- Fork/Join/Merge semantics
fork :: [Stream a] -> IO [Handle a]
fork = mapM (async . execute)

join :: [Handle a] -> IO [a]
join = mapM wait

merge :: [a] -> IO a
merge results = combine results into_unified_context

-- Execution model
execute :: Workflow -> Context -> IO Result
execute workflow ctx = do
  dag <- buildDAG workflow
  sorted <- topologicalSort dag
  levels <- groupByParallelLevel sorted
  results <- for levels $ \level ->
    parallel (map execute level)
  return (final results)
```

6.4 Level 3 Examples

Example 3.1: Comprehensive Library Research

```
(/ctx7 = "stripe" || /ctx7 = "paypal" || /ctx7 = "square") : "compare payment APIs"
```


```
Total: max(5,5,5) + 2 + 10 = 17 min, 22K tokens
vs Sequential: 5+5+5+10 = 25 min, 35K tokens
Speedup: 1.47x, 37% token savings
```

Example 3.2: Full-Stack Research

```
(
  deep-researcher["frontend frameworks"] ||
  deep-researcher["backend frameworks"] ||
  deep-researcher["database options"]
) -> api-architect : "design full-stack architecture"
```


Stream 1: Frontend Research
deep-researcher
Topic: frontend
Researches:
- React vs Vue vs Svelte
- State management
- Build tools
15 min, 20K
Stream 2: Backend Research
deep-researcher
Topic: backend
Researches:
- Node vs Python vs Go
- API frameworks
- Microservices
18 min, 25K
Stream 3: Database Research
deep-researcher
Topic: databases
Researches:
- SQL vs NoSQL
- Scaling strategies
- Caching layers
12 min, 18K
max(15,18,12) = 18 min
Merge Research
2 min, 2K


```
Total: 18 + 2 + 25 = 45 min, 62K tokens
vs Sequential: 15+18+12+25 = 70 min, 100K tokens
Speedup: 1.56x, 38% token savings
```

Example 3.3: Multi-Tool Orchestration

```
(
  /ctx7 = "fastapi" + /ctx7 = "sqlalchemy" ||
  git-genius["analyze repo structure"] ||
  unix-command-master["performance benchmarks"]
) : "optimize existing API"
```

---

7. Level 4: Complex Orchestration

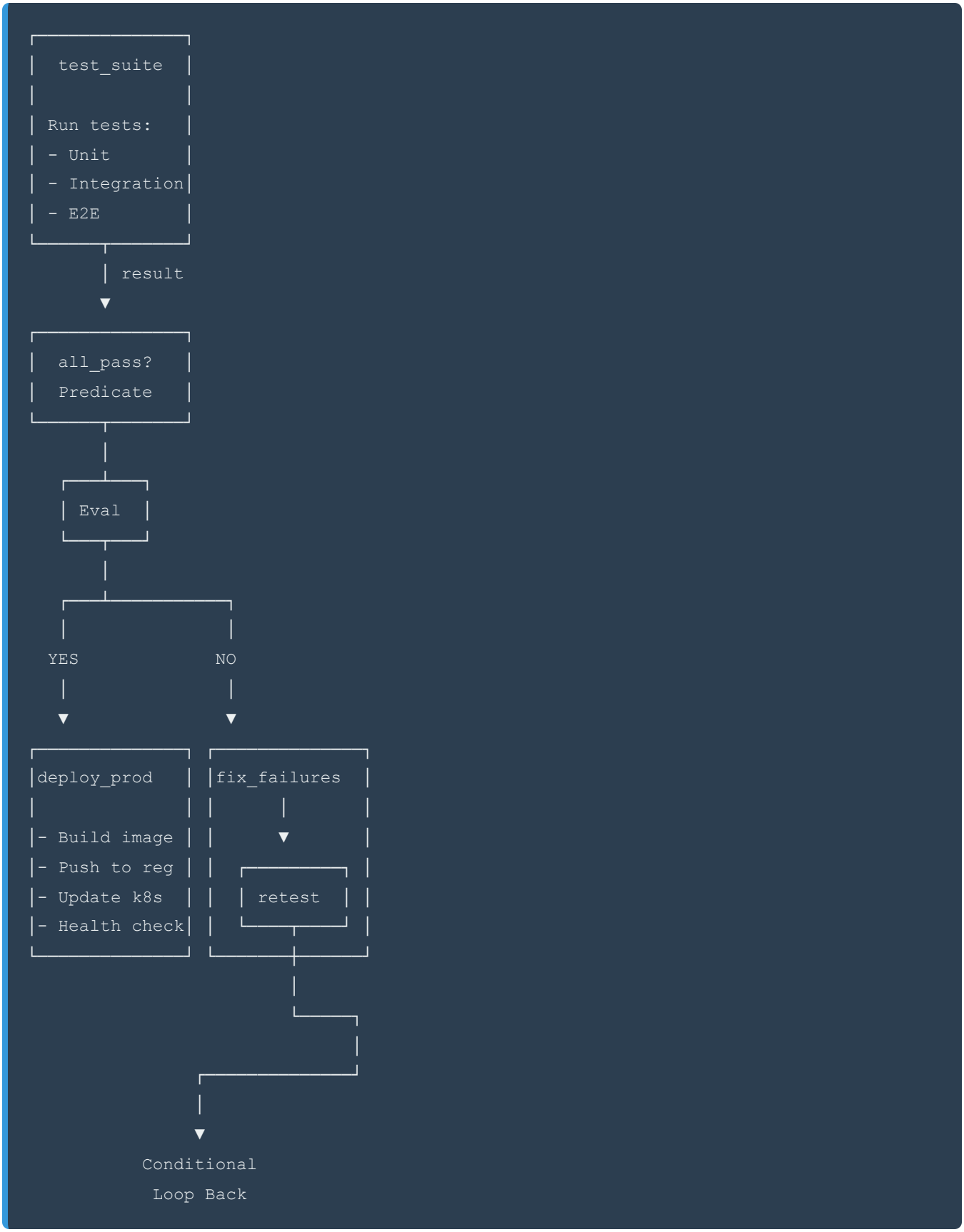
Complexity: High **Execution Model**: DAG with conditional branching **Token Budget**: 80,000-150,000 **Time Estimate**: 45-90 minutes

7.1 Conditional Branching

Concept: Runtime decisions based on intermediate results

```
test_suite -> if(all_pass) deploy_production : (fix_failures -> retest)
```

Visual Model:



Formal Semantics:

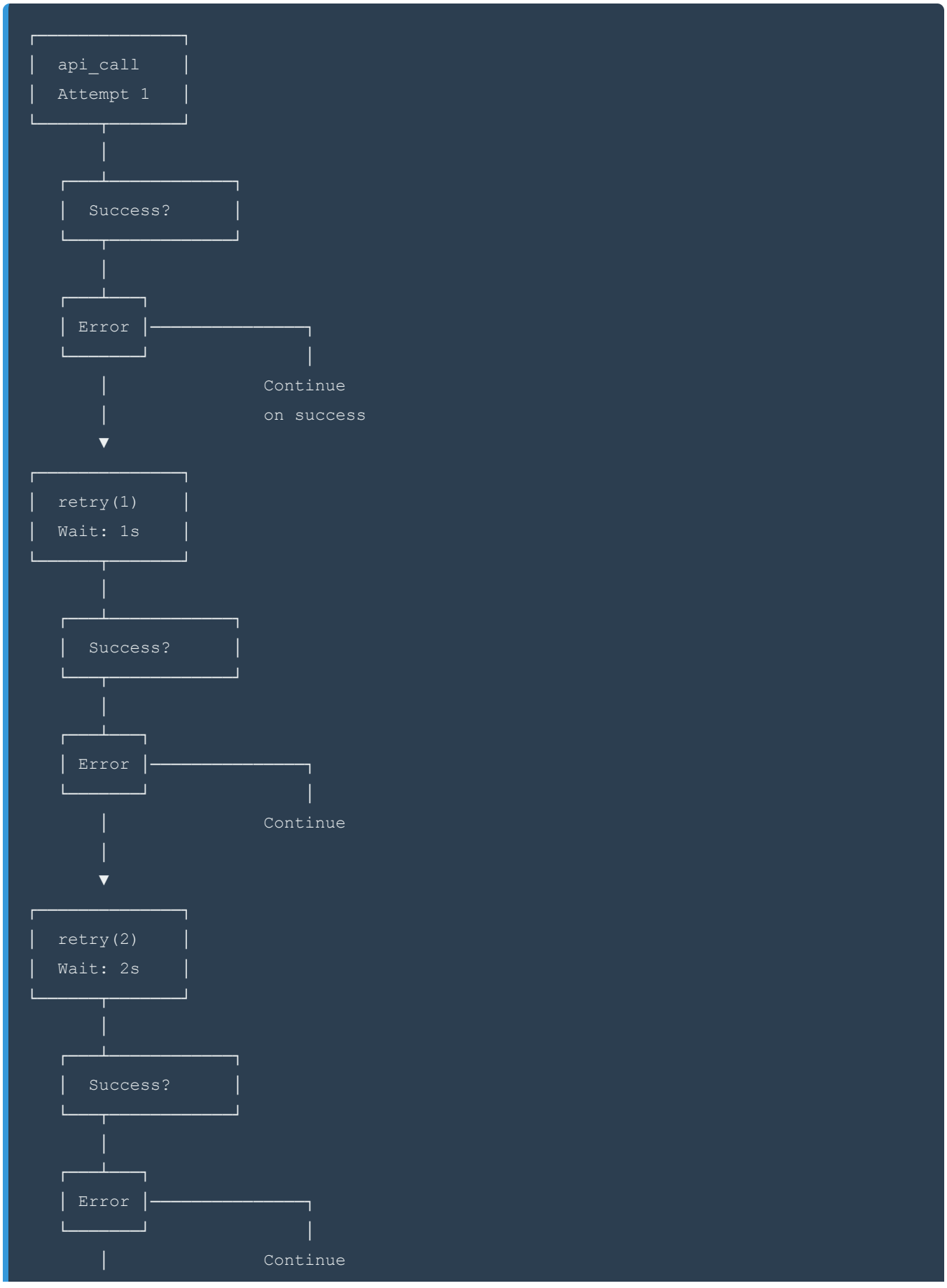
```
-- Conditional workflow
data Conditional a b c where
  If :: Predicate a       -- Condition to evaluate
     -> Workflow a b      -- Then branch
     -> Workflow a c      -- Else branch
     -> Conditional a (Either b c)

-- Evaluation
eval :: Conditional a b c -> a -> IO (Either b c)
eval (If pred thenBranch elseBranch) input = do
  condition <- evaluate pred input
  if condition
    then Left <$> execute thenBranch input
    else Right <$> execute elseBranch input

-- In DSL
test -> if(pass) deploy : fix
  = If (λresult -> result.all_pass)
       (deploy)
       (fix)
```

7.2 Error Handling and Retry Logic

Concept: Graceful degradation and retry strategies

```
api_call -> catch(NetworkError) retry(3, backoff=exponential) : fallback_cache
```

Visual Model:


api_call
Attempt 1
Success?
Error
Continue
on success
retry(1)
Wait: 1s
Success?
Error
Continue
retry(2)
Wait: 2s
Success?
Error
Continue

Formal Semantics:

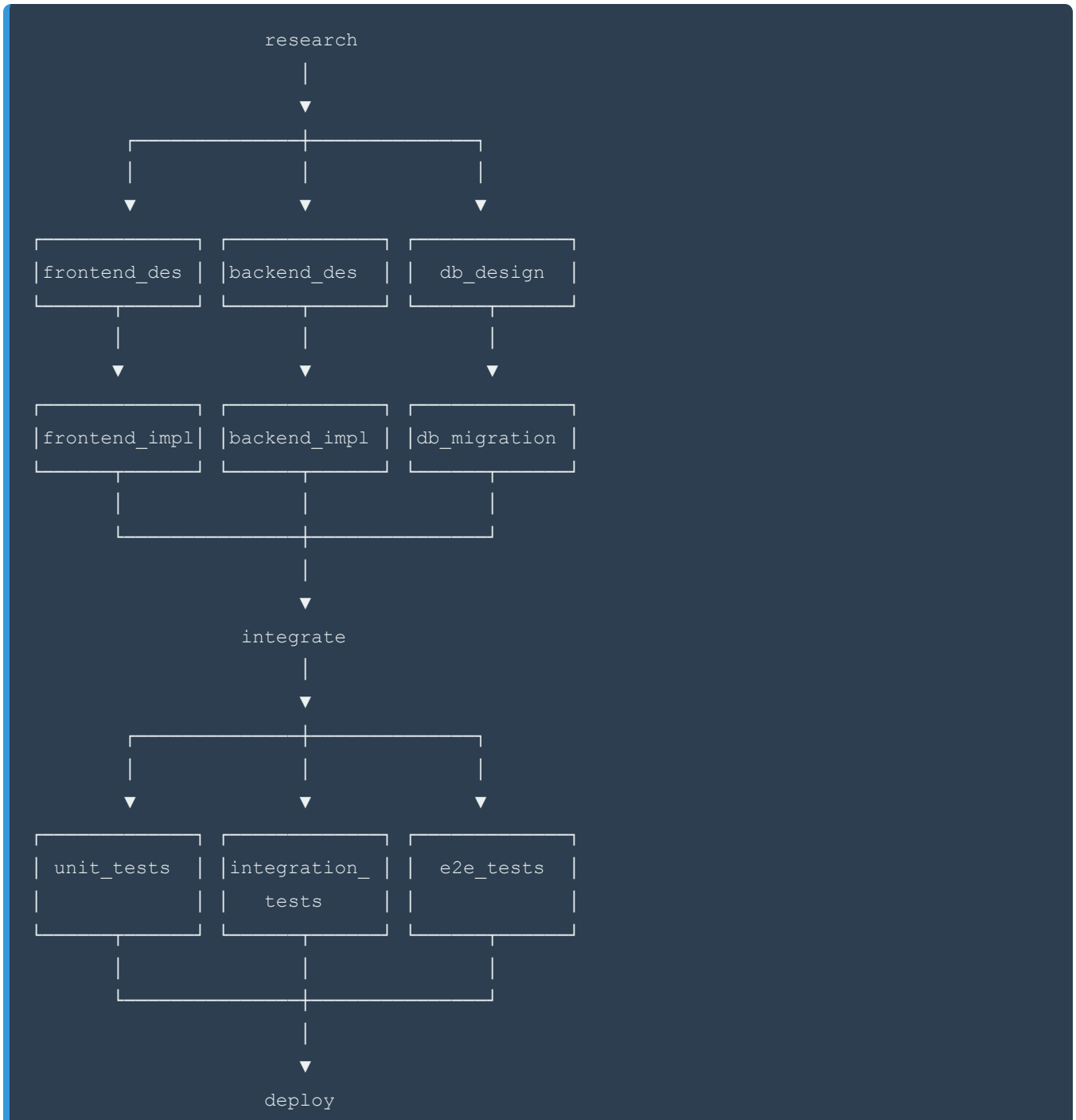
```haskell
-- Retry configuration
data RetryPolicy = RetryPolicy
  { maxAttempts :: Int
  , backoff     :: BackoffStrategy
  , exceptions  :: [ExceptionType]
  }

data BackoffStrategy
  = Constant Seconds
  | Linear Seconds
  | Exponential Seconds
  | Fibonacci Seconds

-- Retry execution
retry :: RetryPolicy -> Workflow a b -> Workflow a b -> Workflow a b
retry policy workflow fallback = go 1
  where
    go attempt = do
      result <- try (execute workflow)
      case result of
        Right value -> return value
        Left err
          | attempt >= maxAttempts policy -> execute fallback
          | err `elem` exceptions policy  -> do
              delay <- backoffDelay (backoff policy) attempt
              threadDelay delay
              go (attempt + 1)
          | otherwise -> throwIO err
```

7.3 Nested Parallel/Sequential Patterns

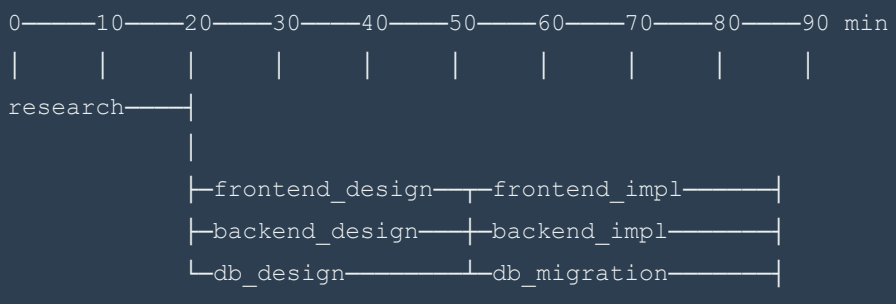
Concept: Complex composition with both parallel and sequential sub-workflows

```
research ->
  (
    (frontend_design -> frontend_impl) ||
    (backend_design -> backend_impl) ||
    (db_design -> db_migration)
  ) ->
integrate ->
  (unit_tests || integration_tests || e2e_tests) ->
deploy
```

Visual Model:



Execution Timeline:



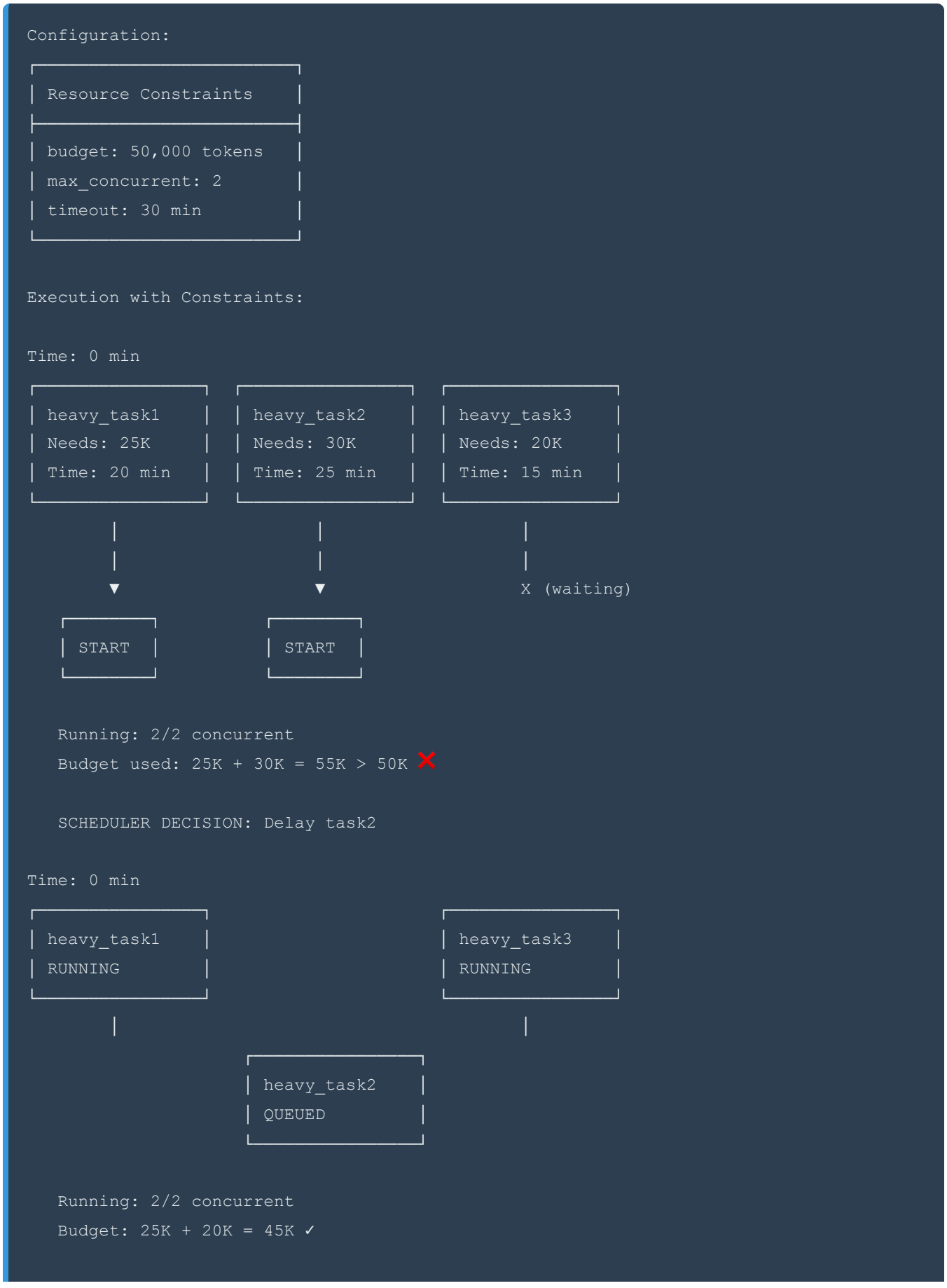
```
                                                  |
                                              integrate
                                                  |
                                          ├─unit_tests─┤
                                          ├─integ_tests─┤
                                          └─e2e_tests──┤
                                                       |
                                                    deploy

Parallel sections save significant time:
  Design phase: max(15,20,18) = 20 min vs 53 min sequential
  Impl phase:   max(25,30,22) = 30 min vs 77 min sequential
  Test phase:   max(10,15,20) = 20 min vs 45 min sequential

Total: 10 + 20 + 30 + 10 + 20 + 15 = 105 min
vs Sequential: 10 + 53 + 77 + 10 + 45 + 15 = 210 min
Speedup: 2x
```

7.4 Resource-Constrained Execution

Concept: Limit parallel execution based on token budget or concurrency limits

```
(heavy_task1 || heavy_task2 || heavy_task3)[
  budget = 50000,
  max_concurrent = 2,
  timeout = 30min
]
```

Visual Model:


Configuration:
Resource Constraints
budget: 50,000 tokens
max_concurrent: 2
timeout: 30 min
Execution with Constraints:
Time: 0 min
heavy_task1
Needs: 25K
Time: 20 min
heavy_task2
Needs: 30K
Time: 25 min
heavy_task3
Needs: 20K
Time: 15 min
X (waiting)
START
START
Running: 2/2 concurrent
Budget used: 25K + 30K = 55K > 50K
SCHEDULER DECISION: Delay task2
Time: 0 min
heavy_task1
RUNNING
heavy_task3
RUNNING
heavy_task2
QUEUED
Running: 2/2 concurrent
Budget: 25K + 20K = 45K ✓

```
Time: 15 min (task3 completes)

┌──────────────────┐
│ heavy_task1      │
│ RUNNING          │  (5 min remaining)
└──────────────────┘

                     ┌──────────────────┐
                     │ heavy_task2      │
                     │ START            │
                     └──────────────────┘


Time: 20 min (task1 completes)

                     ┌──────────────────┐
                     │ heavy_task2      │
                     │ RUNNING          │  (20 min remaining)
                     └──────────────────┘


Time: 40 min (all complete)

Total time: 40 min
vs Unlimited parallel: max(20,25,15) = 25 min
vs Sequential: 60 min

Overhead from constraints: 40 - 25 = 15 min
But stayed within budget ✓
```

Formal Semantics:

```haskell
-- Resource constraints
data Constraints = Constraints
  { tokenBudget    :: Maybe Int
  , maxConcurrent  :: Maybe Int
  , timeout        :: Maybe Duration
  , priority       :: Priority
  }

-- Constrained execution
executeConstrained :: Constraints -> [Workflow a b] -> IO [b]
executeConstrained constraints workflows = do
  scheduler <- newScheduler constraints
  results <- for workflows $ \workflow ->
    schedule scheduler workflow
  await results

-- Scheduling algorithm
schedule :: Scheduler -> Workflow a b -> IO (Async b)
schedule sched workflow = do
  -- Wait for resource availability
  waitForResources sched (resourcesNeeded workflow)

  -- Acquire resources
  resources <- acquireResources sched workflow

  -- Execute with timeout
  asyncWithTimeout (timeout $ constraints sched) $ do
    result <- execute workflow
    releaseResources sched resources
    return result
```

7.5 Dynamic Agent Selection

Concept: Choose agent at runtime based on context or heuristics

```
analyze_code -> select_best_agent(language) -> implement_feature
  where
    select_best_agent("python") = python-specialist
    select_best_agent("rust")   = rust-expert
    select_best_agent(_)        = general-programmer
```

Visual Model:

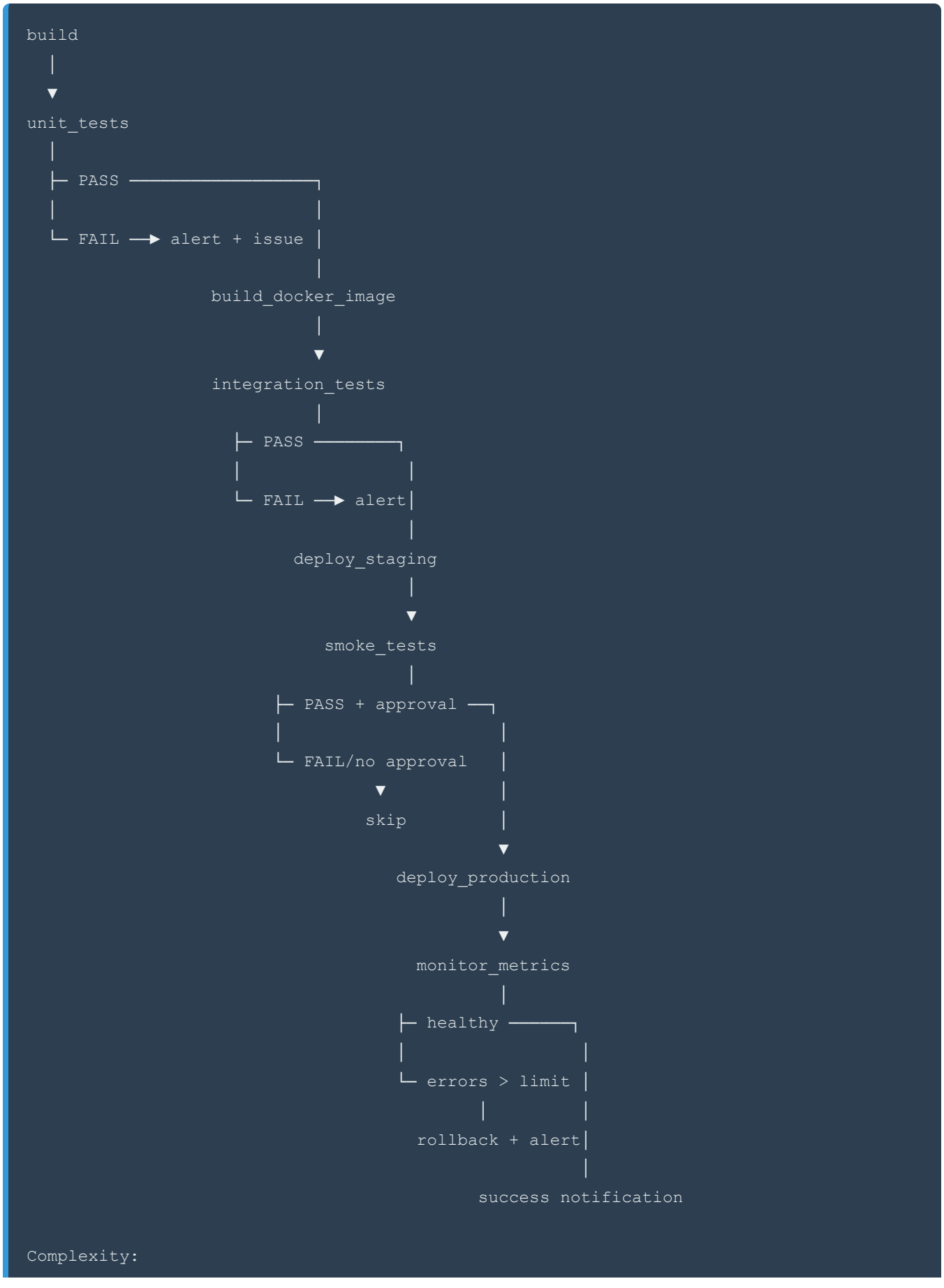

analyze_code
Detects:
- Language
- Framework
- Complexity
analysis_result
select_best_agent
Input: language
Match:
"python" → python-sp
"rust" → rust-exp
"go" → go-expert
"javascript" → js-sp
default → general
selected_agent
Dispatch to chosen agent
py-sp
rust-exp
etc.
implement_feat

```
Example Flow (Python):
  analyze_code("app.py")
    → { language: "python", framework: "fastapi" }
    → select_best_agent("python")
    → python-specialist
    → implement_feature(fastapi_context)
```

7.6 Level 4 Examples

Example 4.1: CI/CD Pipeline with Gates

```
workflow cicd_pipeline {
  build ->
  unit_tests ->
  if(tests_pass) {
    build_docker_image ->
    integration_tests ->
    if(integration_pass) {
      deploy_staging ->
      smoke_tests ->
      if(smoke_pass && approval_received) {
        deploy_production ->
        monitor_metrics ->
        if(error_rate > threshold) {
          rollback_deployment ->
          alert_team
        } else {
          success_notification
        }
      } else {
        skip_production_deployment
      }
    } else {
      alert_failure("Integration tests failed")
    }
  } else {
    alert_failure("Unit tests failed") ->
    create_github_issue
  }
}
```


build
unit_tests
PASS
FAIL
alert + issue
build_docker_image
integration_tests
PASS
FAIL
alert
deploy_staging
smoke_tests
PASS + approval
FAIL/no approval
skip
deploy_production
monitor_metrics
healthy
errors > limit
rollback + alert
success notification

Complexity:

```
  - 7 conditional branches
  - 14 total steps
  - 4 possible end states
  - Time: 60-90 min depending on path
  - Tokens: 80K-120K
```

Example 4.2: Adaptive Research Pipeline

```
workflow adaptive_research(topic) {
  initial_research(topic) ->
  assess_complexity ->

  if(complexity == "high") {
    (
      deep-researcher["academic"] ||
      deep-researcher["industry"] ||
      deep-researcher["open-source"]
    ) ->
    synthesize_comprehensive_report
  } else if(complexity == "medium") {
    (
      /ctx7(related_libraries) ||
      deep-researcher["best-practices"]
    ) ->
    create_practical_guide
  } else {
    quick_reference_lookup ->
    generate_summary
  }
}
```

Example 4.3: Fault-Tolerant Microservice Deployment

```
workflow deploy_microservice(service) {
  (
    build_service ||
    run_security_scan ||
    update_documentation
  ) ->

  integration_tests ->
  catch(TestFailure) {
    retry(3, backoff=exponential) ->
    if(still_failing) {
      rollback ->
      notify_team ->
      create_incident
    }
  } ->

  if(tests_pass) {
    deploy_canary[percentage=10] ->
    monitor_canary[duration=15min] ->

    if(canary_healthy) {
      gradual_rollout[
        steps = [25%, 50%, 100%],
        interval = 10min,
        rollback_on_error = true
      ]
    } else {
      rollback_canary ->
      analyze_failures ->
      create_report
    }
  }
}
```

---

8. Level 5: Workflow Composition

Complexity: Very High **Execution Model**: Higher-order workflows, meta-composition **Token Budget**: 120,000-250,000 **Time Estimate**: 90-180 minutes

8.1 Named Workflows

Concept: Define reusable workflow templates that can be invoked like functions

```
workflow microservice_dev(service_name, domain) {
  research_domain(domain) ->
  (
    design_api(service_name) ||
    design_database(domain) ||
    design_infrastructure
  ) ->
  generate_boilerplate(service_name) ->
  implement_core_logic(domain) ->
  (
    write_unit_tests ||
    write_integration_tests ||
    write_api_tests
  ) ->
  setup_ci_cd ->
  deploy_to_staging ->
  run_acceptance_tests ->
  if(acceptance_pass) {
    deploy_to_production
  } else {
    investigate_failures -> fix_issues -> retry
  }
}

// Invoke workflow
microservice_dev("user-service", "authentication")
microservice_dev("payment-service", "billing")
```

Visual Model:

Formal Semantics:

```
-- Workflow definition
data WorkflowDef params result = WorkflowDef
  { name       :: String
  , parameters :: params
  , body       :: Workflow params result
  , metadata   :: WorkflowMetadata
  }

-- Workflow invocation
invoke :: WorkflowDef params result -> params -> IO result
invoke workflow params = do
  -- Validate parameters
  validated <- validateParams (parameters workflow) params

  -- Create execution context
  context <- createContext workflow validated

  -- Execute workflow body
  result <- executeWorkflow (body workflow) context

  -- Return result
  return result

-- Example
microservice_dev :: WorkflowDef (ServiceName, Domain) Deployment
microservice_dev = WorkflowDef
  { name = "microservice_dev"
  , parameters = (serviceName, domain)
  , body = researchDomain domain
           `andThen` parallel [designAPI serviceName, designDB domain, designInfra]
           `andThen` generateBoilerplate serviceName
           -- ... rest of steps
  , metadata = WorkflowMetadata
      { estimatedTime = 90 * minutes
      , estimatedTokens = 120000
      , tags = ["microservice", "development", "deployment"]
      }
  }
```

8.2 Workflow Composition (Calling Workflows)

Concept: Workflows can invoke other workflows, creating hierarchical composition

```
workflow full_platform_development {
  infrastructure_setup() ->

  parallel [
    microservice_dev("auth-service", "authentication"),
    microservice_dev("user-service", "user-management"),
    microservice_dev("payment-service", "billing"),
    microservice_dev("notification-service", "messaging")
  ] ->

  setup_api_gateway() ->
  setup_service_mesh() ->

  integration_testing_suite() ->

  if(all_tests_pass) {
    deploy_platform() ->
    setup_monitoring() ->
    setup_alerting()
  }
}
```

Visual Model:


LEVEL 0: full_platform_development
infrastructure_setup() (15 min)
LEVEL 1: microservice_dev("auth")
LEVEL 1: microservice_dev("user")
LEVEL 1: microservice_dev("pay")
LEVEL 1: microservice_dev("notif")
Each workflow runs the full microservice pipeline:
research
design
implement
test
deploy
90 min
120K tokens
95 min
125K tokens
100 min
135K tokens
85 min
115K tokens
setup_api_gateway() (10 min)
setup_service_mesh() (15 min)
integration_testing_suite() (30 min)
PASS
FAIL
(exit)
deploy_platform() (20 min)
setup_monitoring() (10 min)
setup_alerting() (5 min)

```
Total Time: 15 + max(90,95,100,85) + 10 + 15 + 30 + 20 + 10 + 5
          = 15 + 100 + 90
          = 205 minutes (~3.4 hours)

vs Sequential microservices: 15 + (90+95+100+85) + 90 = 475 min (~8 hours)
Speedup: 2.3x

Total Tokens: 120K + 125K + 135K + 115K + 50K (other steps)
            = 545K tokens
```

Hierarchical Call Graph:

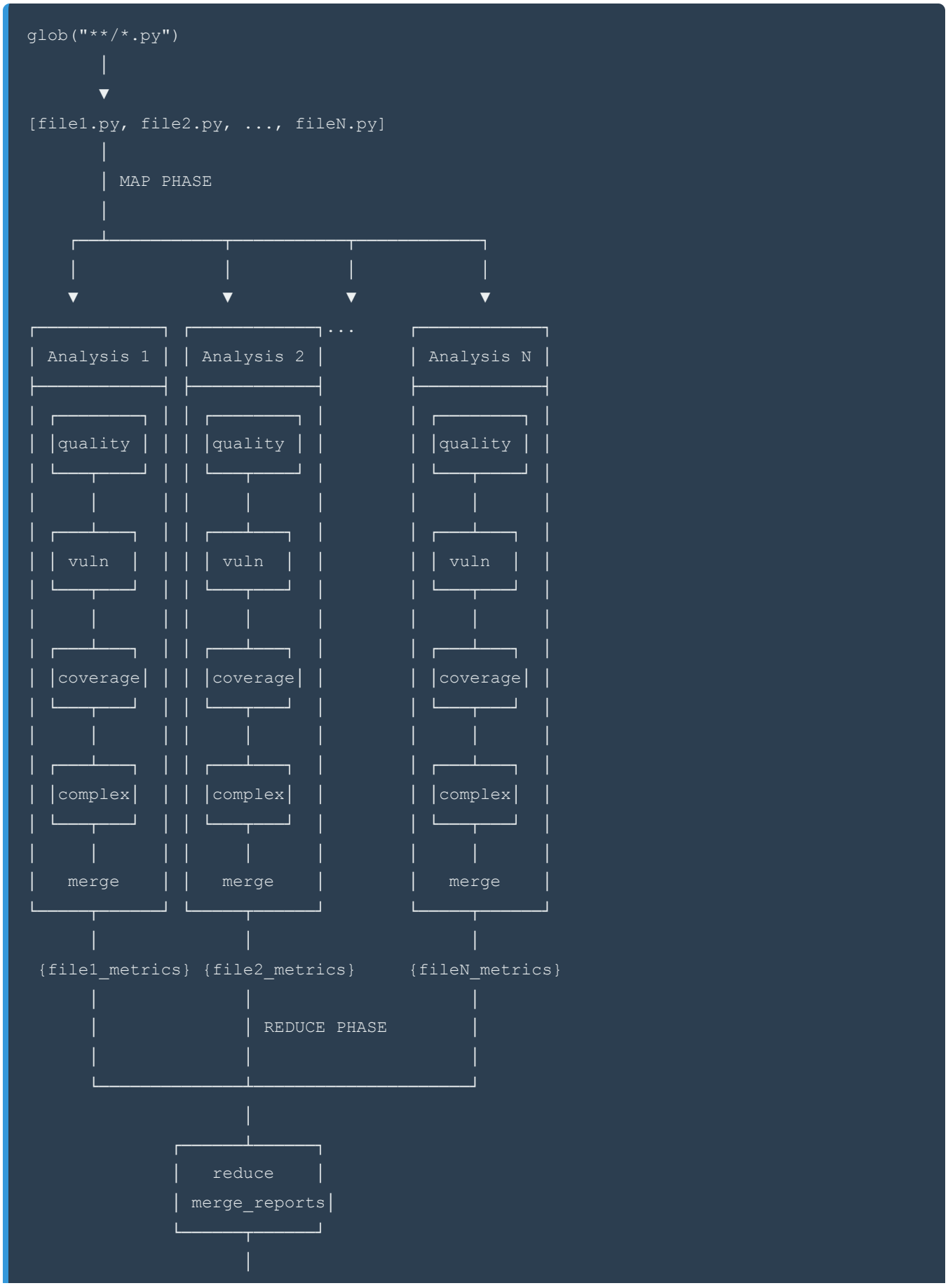
```
full_platform_development
│
├─ infrastructure_setup
│
├─ microservice_dev("auth-service")
│  ├─ research_domain("authentication")
│  ├─ design_api("auth-service")
│  ├─ design_database("authentication")
│  ├─ design_infrastructure
│  ├─ generate_boilerplate("auth-service")
│  └─ ... (full pipeline)
│
├─ microservice_dev("user-service")
│  └─ ... (full pipeline)
│
├─ microservice_dev("payment-service")
│  └─ ... (full pipeline)
│
├─ microservice_dev("notification-service")
│  └─ ... (full pipeline)
│
├─ setup_api_gateway
├─ setup_service_mesh
├─ integration_testing_suite
├─ deploy_platform
├─ setup_monitoring
└─ setup_alerting

Depth: 3 levels
Width: 4 parallel workflows at level 1
Total Nodes: 50+ individual steps
```

8.3 Map-Reduce Patterns

Concept: Apply a workflow to a collection of items in parallel, then reduce

```
workflow analyze_repository(repo_path) {
  files = glob(repo_path + "/**/*.py")

  // MAP: Analyze each file in parallel
  analyses = map(files, |file| {
    (
      code_quality_check(file) ||
      security_vulnerability_scan(file) ||
      test_coverage_analysis(file) ||
      complexity_metrics(file)
    ) -> aggregate_file_metrics(file)
  })

  // REDUCE: Combine all analyses
  repository_report = reduce(analyses, merge_reports)

  // Final synthesis
  generate_recommendations(repository_report) ->
  create_pr_with_fixes(repository_report) ->
  notify_team(repository_report)
}
```

Visual Model:


glob("**/*.py")
[file1.py, file2.py, ..., fileN.py]
MAP PHASE
Analysis 1
Analysis 2
Analysis N
quality
vuln
coverage
complex
merge
{file1_metrics}
{file2_metrics}
{fileN_metrics}
REDUCE PHASE
reduce
merge_reports

```
            repository_report
                    |
        ┌───────────┼───────────┐
        |           |           |
        ▼           ▼           ▼
  generate_rec   create_PR   notify_team

Time Analysis (100 files):
  Map phase (parallel): max(file_times) ≈ 2 min
  Reduce phase: O(log N) ≈ 1 min
  Final steps: 5 min
  Total: ~8 min

vs Sequential: 100 * 2 = 200 min
Speedup: 25x
```

Formal Semantics:

```
-- Map operation
map :: [a] -> (a -> b) -> IO [b]
map items f = parallel (fmap f items)

-- Reduce operation
reduce :: [a] -> (a -> a -> a) -> a
reduce []     _ = error "Cannot reduce empty list"
reduce [x]    _ = x
reduce (x:xs) f = f x (reduce xs f)

-- Map-reduce combinator
mapReduce :: [a] -> (a -> b) -> (b -> b -> b) -> IO b
mapReduce items mapFn reduceFn = do
  mapped <- map items mapFn
  return (reduce mapped reduceFn)

-- Example usage
analyzeRepository :: FilePath -> IO Report
analyzeRepository repo = do
  files <- glob (repo </> "**/*.py")

  mapReduce files
    (\file -> analyzeFile file)  -- Map function
    (\r1 r2 -> mergeReports r1 r2)  -- Reduce function
```

8.4 Context Sharing and Accumulation

Concept: Build shared context progressively, pass through workflow pipeline

```
workflow api_development_with_context {
  // Build comprehensive context
  ctx = empty_context

  ctx = ctx + /ctx7("fastapi")
  ctx = ctx + /ctx7("postgresql")
  ctx = ctx + /ctx7("redis")
  ctx = ctx + /ctx7("pydantic")

  // All agents share accumulated context
  deep-researcher[ctx] : "research best practices" ->
  api-architect[ctx] : "design RESTful API" ->
  practical-programmer[ctx] : "implement with best practices" ->
  test-engineer[ctx] : "write comprehensive tests" ->
  docs-generator[ctx] : "document API"

  // Context contains all library knowledge throughout pipeline
}
```

Visual Model:

```
CONTEXT ACCUMULATION:

ctx₀ = ∅ (empty)
  │
  ├─ /ctx7("fastapi")
  │
ctx₁ = { fastapi_docs }
  │
  ├─ /ctx7("postgresql")
  │
ctx₂ = { fastapi_docs, postgresql_docs }
  │
  ├─ /ctx7("redis")
  │
ctx₃ = { fastapi_docs, postgresql_docs, redis_docs }
  │
  ├─ /ctx7("pydantic")
  │
ctx₄ = { fastapi_docs, postgresql_docs, redis_docs, pydantic_docs }


WORKFLOW WITH SHARED CONTEXT:

┌──────────────────┐
│ deep-researcher  │
│ Context: ctx₄    │
│ Has access to:   │
│ - fastapi ✓      │
│ - postgresql ✓   │
│ - redis ✓        │
│ - pydantic ✓     │
└────────┬─────────┘
         │ research_output + ctx₄
         ▼
┌──────────────────┐
│ api-architect    │
│ Context: ctx₄    │
│ Uses:            │
│ - Research ✓     │
│ - Library docs ✓ │
└────────┬─────────┘
         │ api_design + ctx₄
         ▼
```

Formal Semantics:

```
-- Context type
data Context = Context
  { libraryDocs  :: Map LibraryName Documentation
  , customSkills :: [Skill]
  , environment  :: Environment
  , artifacts    :: [Artifact]
  }

-- Context accumulation
instance Monoid Context where
  mempty = Context Map.empty [] defaultEnv []

  mappend ctx1 ctx2 = Context
    { libraryDocs  = Map.union (libraryDocs ctx1) (libraryDocs ctx2)
    , customSkills = customSkills ctx1 ++ customSkills ctx2
    , environment  = mergeEnv (environment ctx1) (environment ctx2)
    , artifacts    = artifacts ctx1 ++ artifacts ctx2
    }

-- Workflow with context
type WorkflowWithContext a b = Context -> a -> IO (b, Context)

-- Threading context through pipeline
threadContext :: [WorkflowWithContext a a] -> Context -> a -> IO (a, Context)
threadContext [] ctx input = return (input, ctx)
threadContext (w:ws) ctx input = do
  (output, ctx') <- w ctx input
  threadContext ws ctx' output
```

8.5 Parameterized Workflows

Concept: Workflows with type parameters and generic implementations

```
// Generic CRUD workflow generator
workflow crud_service⟨Entity⟩(entity_spec: EntitySpec) {
  generate_schema(entity_spec) ->
  generate_migrations(entity_spec) ->

  parallel [
    generate_crud_handlers(entity_spec),
    generate_validators(entity_spec),
    generate_serializers(entity_spec)
  ] ->

  generate_tests(entity_spec) ->
  generate_documentation(entity_spec)
}

// Instantiate for specific entities
user_service = crud_service⟨User⟩({
  fields: [name, email, password_hash],
  indexes: [email],
  validations: [email_format, password_strength]
})

product_service = crud_service⟨Product⟩({
  fields: [name, price, inventory],
  indexes: [name, price],
  validations: [price_positive, inventory_non_negative]
})
```

Visual Model:


GENERIC WORKFLOW: crud_service(T)
Type Parameter: T extends Entity
Input: EntitySpec<T>
- fields: [Field]
- indexes: [Index]
- validations: [Validation]
Steps (parameterized over T):
1. generate_schema<T>
2. generate_migrations<T>
3. parallel [
generate_crud_handlers<T>,
generate_validators<T>,
generate_serializers<T>
]
4. generate_tests<T>
5. generate_documentation<T>
SPECIALIZATION 1
T = User
EntitySpec:
- name: String
- email: String
- password: Hash
Generates:
- UserSchema
- UserHandlers
- UserTests
- UserDocs
SPECIALIZATION 2
T = Product
EntitySpec:
- name: String
- price: Decimal
- inventory: Int
Generates:
- ProductSchema
- ProductHandlers
- ProductTests
- ProductDocs
Reusability:

```
  - Define once, instantiate many
  - Type-safe specialization
  - Consistent structure across entities
  - Reduced duplication
```

8.6 Level 5 Examples

Example 5.1: Multi-Repo Monorepo Migration

```
workflow migrate_to_monorepo(repos: [Repository]) {
  // Map: Process each repo independently
  processed_repos = map(repos, |repo| {
    analyze_dependencies(repo) ||
    update_import_paths(repo) ||
    extract_shared_code(repo) ->
    create_migration_plan(repo)
  })

  // Merge shared code
  shared_libs = reduce(
    map(processed_repos, extract_shared),
    deduplicate_and_merge
  )

  // Setup monorepo structure
  setup_workspace() ->
  create_package_structure(shared_libs) ->

  // Reduce: Integrate all repos
  integrate_repos = reduce(processed_repos, |acc, repo| {
    move_to_monorepo(repo) ->
    update_dependencies(repo, acc) ->
    run_integration_tests(repo, acc)
  })

  // Final validation
  run_full_test_suite() ->
  update_ci_cd() ->
  generate_migration_report()
}
```

Example 5.2: Progressive Feature Rollout Workflow

```
workflow progressive_rollout(feature, stages) {
  // Initial deployment
  deploy_to_dev(feature) ->
  dev_smoke_tests ->

  // Progressive rollout through stages
  for stage in stages {
    deploy_percentage(feature, stage.percentage) ->
    monitor_metrics(feature, stage.duration) ->

    if(metrics_healthy(stage.thresholds)) {
      log_success(stage) ->
      if(stage.percentage == 100) {
        complete_rollout(feature)
      } else {
        continue  // Next stage
      }
    } else {
      rollback_feature(feature) ->
      analyze_failures(feature) ->
      alert_team(feature, stage) ->
      abort_rollout
    }
  }
}

// Invoke with canary deployment strategy
progressive_rollout("new-algorithm", [
  { percentage: 1,  duration: 30min, thresholds: {...} },
  { percentage: 5,  duration: 1hour, thresholds: {...} },
  { percentage: 25, duration: 4hours, thresholds: {...} },
  { percentage: 100, duration: 24hours, thresholds: {...} }
])
```

Example 5.3: Polyglot Microservices Platform

```
workflow build_polyglot_platform {
  // Context accumulation for each language
  go_ctx     = /ctx7("golang") + /ctx7("gin")
  python_ctx = /ctx7("python") + /ctx7("fastapi")
  rust_ctx   = /ctx7("rust") + /ctx7("actix-web")

  // Build services in parallel, each with appropriate context
  parallel [
    microservice_dev[go_ctx]("gateway-service", "routing"),
    microservice_dev[python_ctx]("ml-service", "predictions"),
    microservice_dev[rust_ctx]("streaming-service", "realtime"),
    microservice_dev[python_ctx]("auth-service", "authentication")
  ] ->

  // Integration layer
  setup_service_mesh() ->
  configure_mutual_tls() ->
  setup_distributed_tracing() ->

  // Testing
  (
    contract_testing ||
    load_testing ||
    chaos_engineering
  ) ->

  deploy_platform()
}
```

---

9. Level 6: Meta-Programming

Complexity: Extreme **Execution Model**: Abstract workflow generation, type-level computation
Token Budget: 200,000+ **Time Estimate**: 3+ hours

9.1 Category Theory: Functors

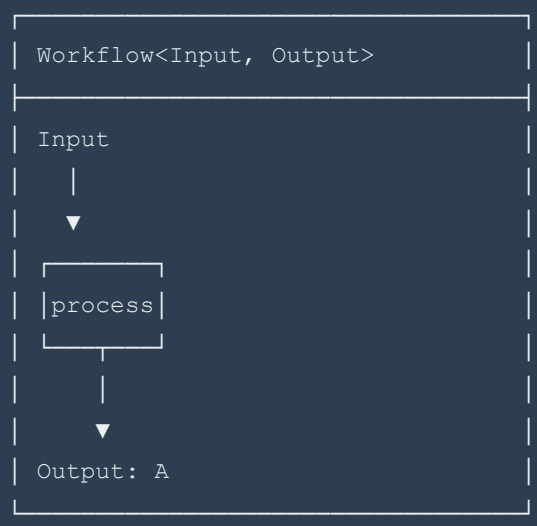
Concept: Map over workflow structures while preserving composition

```
// Functor: Transform workflow outputs
functor F<A> = Workflow<_, A>

fmap: (A -> B) -> F<A> -> F<B>
fmap(transform, workflow) = workflow -> map_output(transform)

// Example: Transform all outputs to JSON
json_functor = fmap(to_json, _)

api_workflow :: Workflow<Request, ApiResponse>
json_workflow :: Workflow<Request, JSON>
json_workflow = json_functor(api_workflow)
```

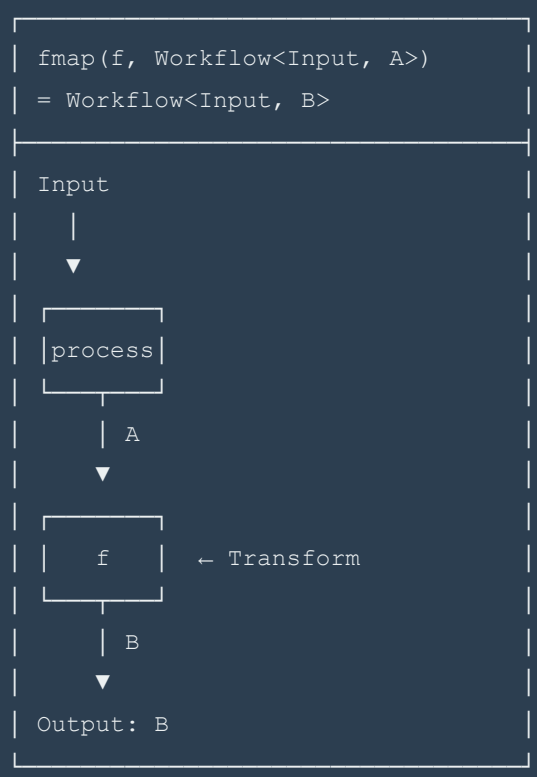
Visual Model:

FUNCTOR MAPPING:

Original Workflow:



Apply Functor with f: A -> B:



Functor Laws:

1. Identity: fmap(id) = id
2. Composition: fmap(f ∘ g) = fmap(f) ∘ fmap(g)

Visual proof of composition law:

```
        workflow
           |
     fmap(g ∘ f)
           |
           ▼
        result₁

        workflow
           |
        fmap(g)
           |
        fmap(f)
           |
           ▼
        result₂

result₁ = result₂ (law holds)
```

Example: Logging Functor

```
// Add logging to any workflow
logging_functor = fmap(|result| {
  log("Result:", result)
  return result
}, _)

// Transform workflow
basic_workflow: research -> design
logged_workflow: logging_functor(basic_workflow)

// Execution trace:
// research -> log(research_output) -> design -> log(design_output)
```

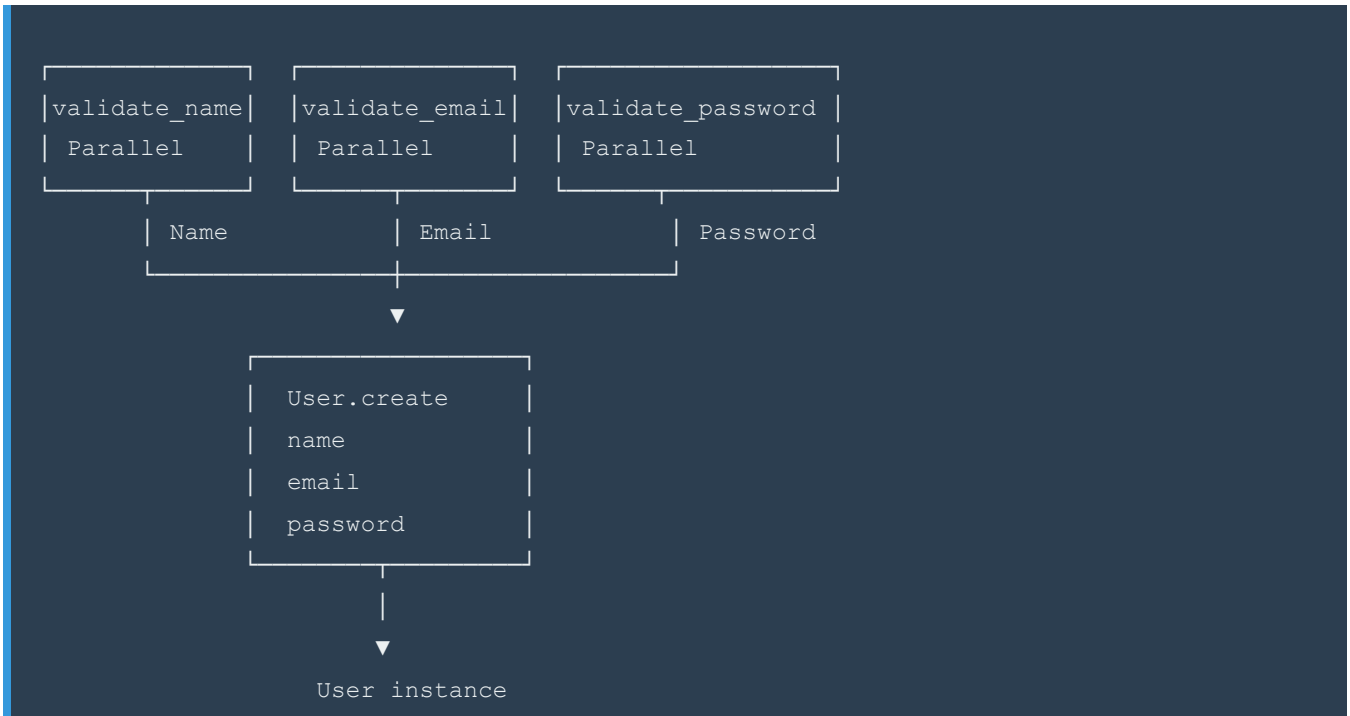
9.2 Applicative Functors

Concept: Apply workflows with wrapped functions to wrapped values

```
// Applicative functor
class Applicative F where
  pure: A -> F<A>
  (<*>): F<(A -> B)> -> F<A> -> F<B>

// Example: Parallel validation
validate_user = pure(User.create)
  <*> validate_name(name_input)
  <*> validate_email(email_input)
  <*> validate_password(password_input)

// All validations run in parallel
// If all succeed, User.create is called with validated values
// If any fails, entire workflow fails
```

Visual Model:

```
APPLICATIVE FUNCTOR:

Step 1: Lift constructor into workflow
pure(User.create) :: Workflow<(), Name -> Email -> Password -> User>

Step 2: Apply to first argument
pure(User.create) <*> validate_name
  :: Workflow<NameInput, Email -> Password -> User>

┌──────────────────┐
│ validate_name    │
│ Input: string    │
│ Output: Name     │
└──────────────────┘
        │ Name
        ▼
┌──────────────────┐
│ User.create      │
│ Partial: Email   │
│    -> Password   │
│    -> User       │
└──────────────────┘

Step 3: Apply to second argument
... <*> validate_email
  :: Workflow<EmailInput, Password -> User>

┌──────────────────┐   ┌──────────────────┐
│ validate_name    │   │ validate_email   │
│ Parallel         │   │ Parallel         │
└──────────┬───────┘   └──────────┬───────┘
        │ Name                 │ Email
        └──────────┬───────────┘
                   ▼
        ┌──────────────────┐
        │ User.create      │
        │ Partial: Pass    │
        │    -> User       │
        └──────────────────┘

Step 4: Apply to third argument
... <*> validate_password
  :: Workflow<PasswordInput, User>
```




```
Applicative Laws:
1. Identity:      pure(id) <*> v = v
2. Composition:   pure(∘) <*> u <*> v <*> w = u <*> (v <*> w)
3. Homomorphism:  pure(f) <*> pure(x) = pure(f(x))
4. Interchange:   u <*> pure(y) = pure(λf. f(y)) <*> u
```

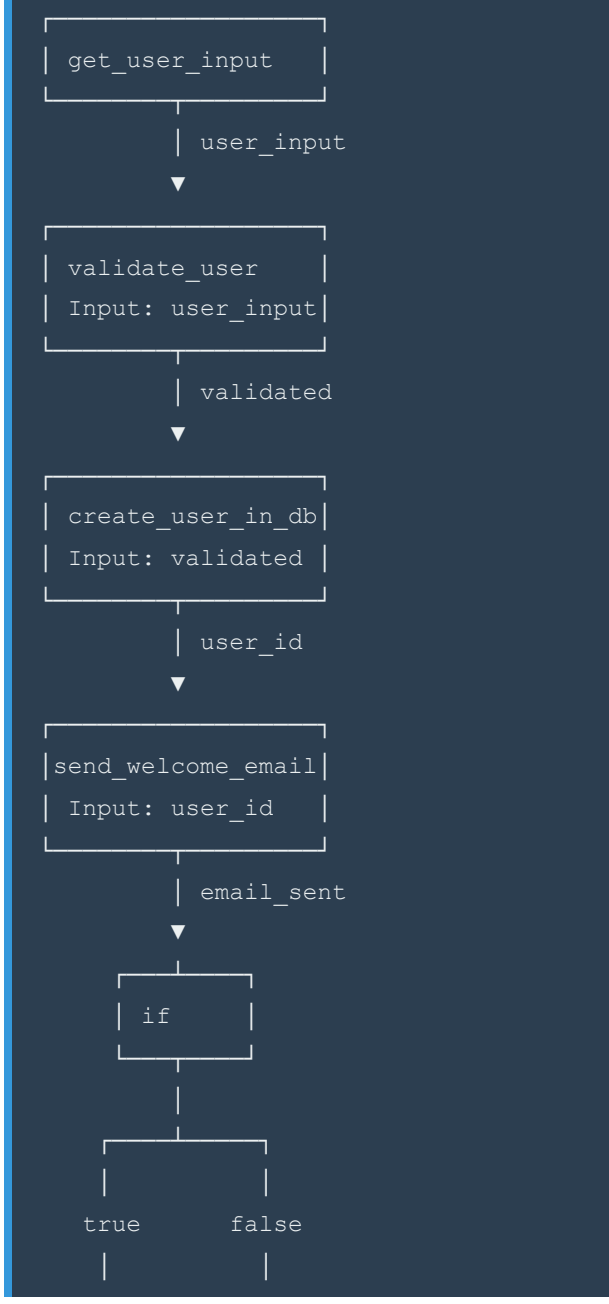
9.3 Monads and Do-Notation

Concept: Sequential composition with implicit context threading

```
// Monad interface
class Monad M where
  return: A -> M<A>
  (>>=): M<A> -> (A -> M<B>) -> M<B>  // bind operator

// Do-notation for workflows
workflow user_registration = do {
  user_input <- get_user_input

  validated <- validate_user(user_input)

  user_id <- create_user_in_db(validated)

  email_sent <- send_welcome_email(user_id)

  if email_sent then
    return success(user_id)
  else
    log_email_failure(user_id) >>
    return partial_success(user_id)
}
```

Visual Model:

```
MONADIC COMPOSITION:

Without do-notation (explicit bind):
get_user_input >>= λuser_input ->
validate_user(user_input) >>= λvalidated ->
create_user_in_db(validated) >>= λuser_id ->
send_welcome_email(user_id) >>= λemail_sent ->
if email_sent
  then return success(user_id)
  else log_email_failure(user_id) >> return partial_success(user_id)

Visual execution flow:
```



```
    ▼          ▼
success    log_failure
  (id)          |
                ▼
         partial_success
              (id)

Context Threading (implicit in monad):
Each step automatically receives:
  - Previous results
  - Error state
  - Execution context
  - Resource handles

Monad Laws:
1. Left Identity:   return a >>= f  ≡  f a
2. Right Identity:  m >>= return    ≡  m
3. Associativity:   (m >>= f) >>= g ≡  m >>= (λx. f x >>= g)
```

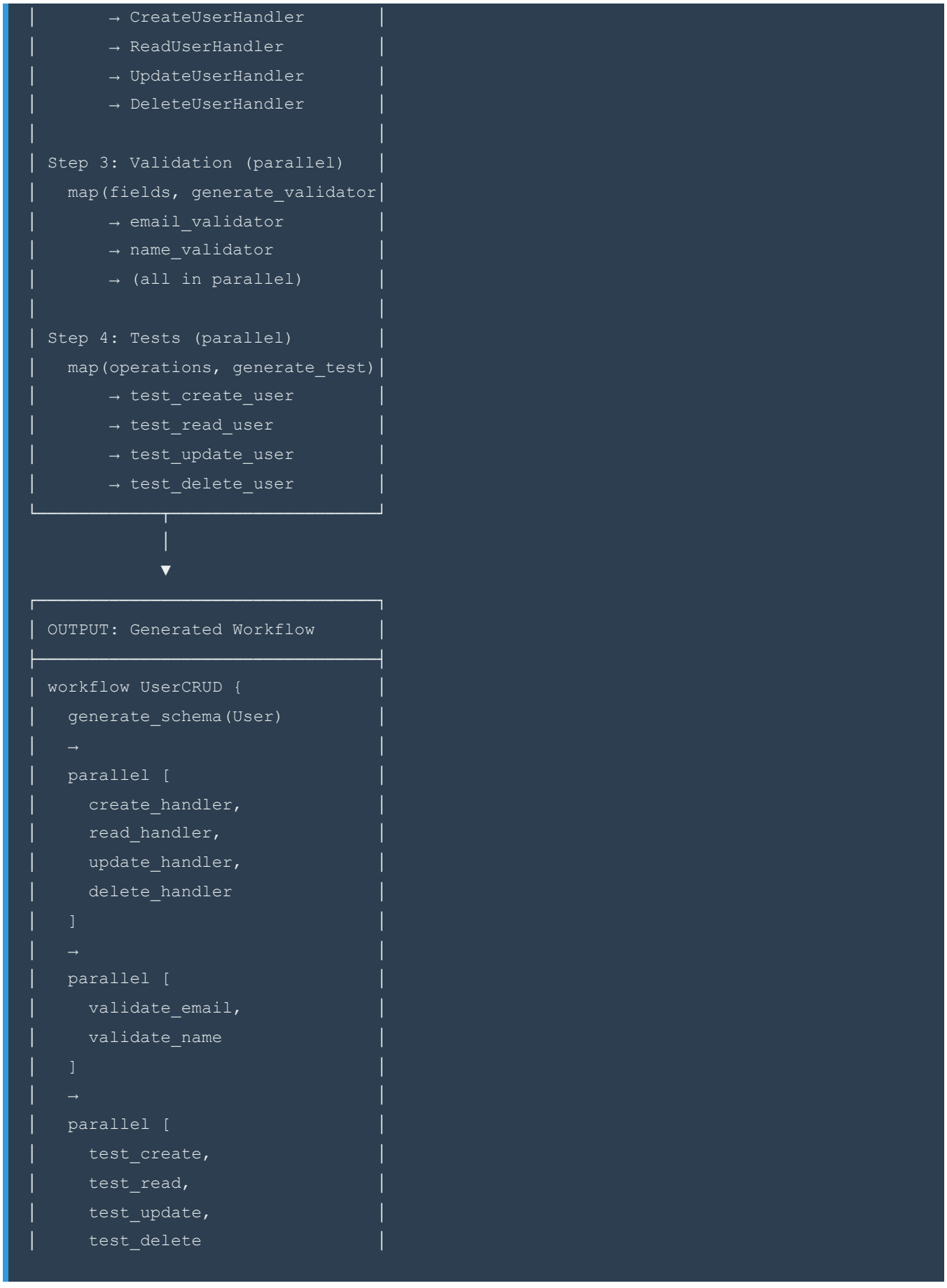
9.4 Workflow Generators (Meta-Workflows)

Concept: Functions that generate workflows based on specifications

```
// Meta-workflow: Generate CRUD workflow for any entity
meta_workflow generate_crud⟨Entity⟩(spec: EntitySpec) -> Workflow {
  // Analyze entity specification
  fields = analyze_fields(spec)
  relations = analyze_relations(spec)
  constraints = analyze_constraints(spec)

  // Generate workflow steps dynamically
  workflow = empty_workflow()

  // Add schema generation
  workflow = workflow.add_step(
    generate_schema_step(Entity, fields, constraints)
  )

  // Add CRUD handlers for each operation
  for operation in [Create, Read, Update, Delete] {
    workflow = workflow.add_step(
      generate_handler_step(Entity, operation, fields)
    )
  }

  // Add validation for each field
  validators = parallel(
    map(fields, |field| generate_validator(field))
  )
  workflow = workflow.add_step(validators)

  // Add tests
  test_cases = generate_test_cases(Entity, [Create, Read, Update, Delete])
  workflow = workflow.add_step(
    parallel(map(test_cases, generate_test))
  )

  return workflow
}

// Generate concrete workflows
UserWorkflow = generate_crud⟨User⟩(UserSpec)
ProductWorkflow = generate_crud⟨Product⟩(ProductSpec)
OrderWorkflow = generate_crud⟨Order⟩(OrderSpec)
```

Visual Model:


META-WORKFLOW GENERATOR:
Input: EntitySpec
Entity: User
Fields:
- id: UUID
- name: String
- email: Email
- created_at: DateTime
Relations:
- posts: [Post]
- profile: Profile
Constraints:
- email: unique
- name: non-empty
ANALYSIS PHASE
analyze_fields()
→ [id, name, email, ...]
analyze_relations()
→ [posts (1:N), profile (1:1)]
analyze_constraints()
→ [email.unique, name.nonempty]
GENERATION PHASE
Step 1: Schema Generation
generate_schema(User)
→ SQL/ORM schema
Step 2: CRUD Handlers (loop)
for op in CRUD:
generate_handler(User, op)


→ CreateUserHandler
→ ReadUserHandler
→ UpdateUserHandler
→ DeleteUserHandler
Step 3: Validation (parallel)
map(fields, generate_validator
→ email_validator
→ name_validator
→ (all in parallel)
Step 4: Tests (parallel)
map(operations, generate_test)
→ test_create_user
→ test_read_user
→ test_update_user
→ test_delete_user
OUTPUT: Generated Workflow
workflow UserCRUD {
generate_schema(User)
→
parallel [
create_handler,
read_handler,
update_handler,
delete_handler
]
→
parallel [
validate_email,
validate_name
]
→
parallel [
test_create,
test_read,
test_update,
test_delete

```
|   ]                                 |
| }                                   |
└─────────────────────────────────────┘

Code Generation (from meta-workflow):
  Input:  EntitySpec (declarative)
  Output: Executable Workflow (imperative)

  Generates:
    - Database schema (SQL/ORM)
    - API handlers (Python/Rust/Go)
    - Validators (regex, type checks)
    - Tests (pytest/junit/cargo test)
    - Documentation (OpenAPI/JSDoc)
```

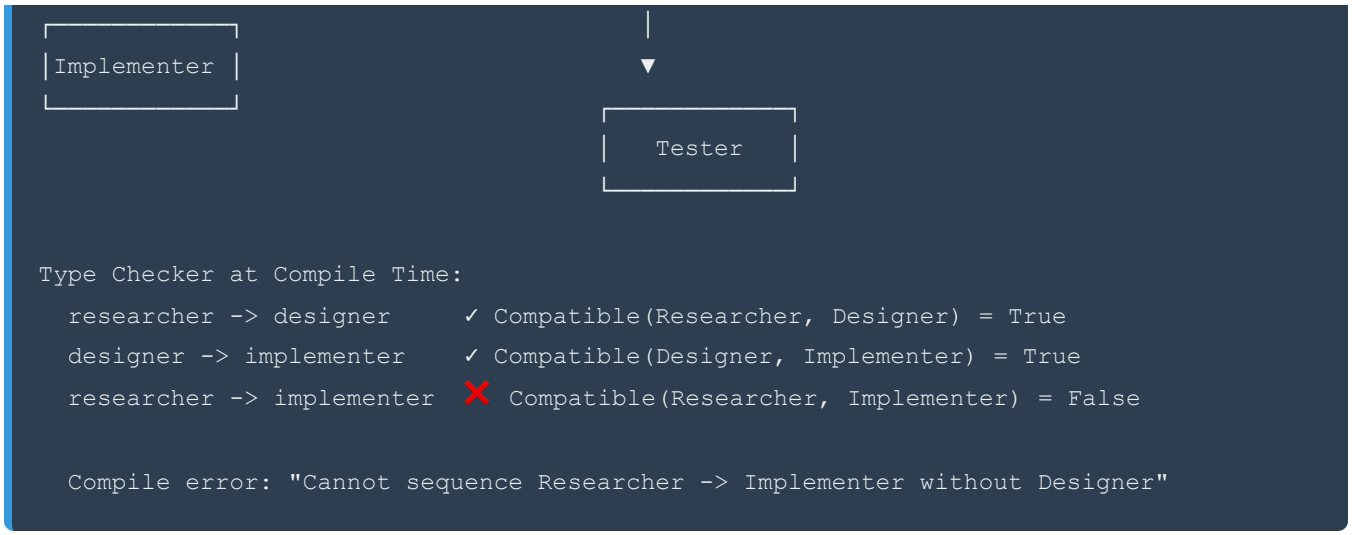
9.5 Type-Level Computation

Concept: Compute types at compile-time to ensure correctness

```
// Dependent types: return type depends on input value
type family WorkflowResult(n: Nat) where
  WorkflowResult(0) = Empty
  WorkflowResult(1) = Single Result
  WorkflowResult(n) = Vector n Result

// Type-safe pipeline length
pipeline(n: Nat): Vector n Agent -> Input -> WorkflowResult(n)
pipeline([a1, a2, a3], input) :: WorkflowResult(3)
  = Vector 3 Result

// Compile-time verification
type family Compatible(a: AgentType, b: AgentType) where
  Compatible(Researcher, Designer)   = True
  Compatible(Designer, Implementer)  = True
  Compatible(Implementer, Tester)    = True
  Compatible(_, _)                   = False

// Type error if incompatible agents are sequenced
safe_sequence: (Compatible(A, B) ~ True) => A -> B -> Workflow
safe_sequence(researcher, implementer)  -- ❌ Type error!
safe_sequence(researcher, designer)     -- ✓ OK
```

Visual Model:

```
TYPE-LEVEL COMPUTATION:

Example: Type-safe pipeline construction

Level 1: Value Level (runtime)
┌──────────────────────────────────┐
│ agents = [researcher, designer,  │
│           implementer]           │
│                                  │
│ result = pipeline(agents, input) │
└──────────────────────────────────┘

Level 2: Type Level (compile-time)
┌──────────────────────────────────┐
│ agents :: Vector 3 Agent         │
│           ^^^^^^ type parameter  │
│                                  │
│ pipeline :: Vector n Agent       │
│          -> Input                │
│          -> WorkflowResult(n)    │
│                                  │
│ WorkflowResult(3)                │
│   = Vector 3 Result              │
│                                  │
│ Type checker ensures:            │
│   - Correct number of results    │
│   - Type safety across pipeline  │
└──────────────────────────────────┘

Compatibility Type Function:

┌────────────┐    Compatible?    ┌────────────┐
│ Researcher │──────────────────►│  Designer  │
└────────────┘       Yes         └────────────┘
      │                               │
      │ Compatible?                   │ Compatible?
      │                               │
      ▼                               ▼
      No                        ┌────────────┐
      │                         │Implementer │
      │                         └────────────┘
      │                               │
      ▼                               │ Compatible?
```



9.6 Workflow Optimization

Concept: Automatically optimize workflow execution plans

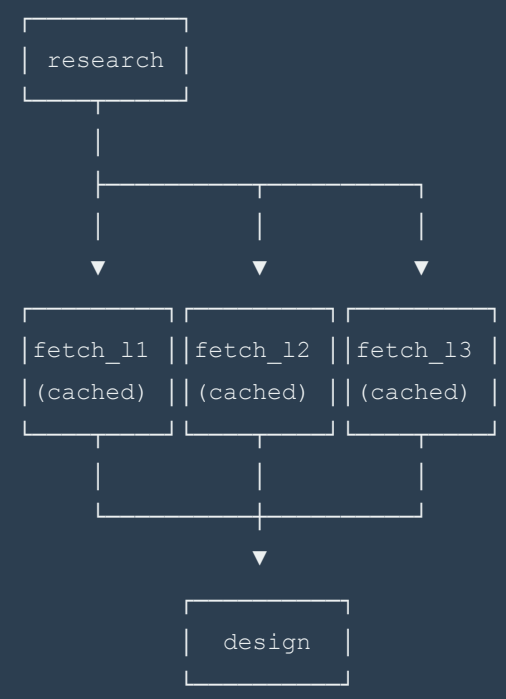
```
// Workflow optimizer
optimize: Workflow -> Workflow
optimize(workflow) = {
  dag = build_dag(workflow)

  // Optimization passes
  dag = detect_parallelism(dag)
  dag = eliminate_redundancy(dag)
  dag = reorder_for_locality(dag)
  dag = insert_caching(dag)
  dag = balance_load(dag)

  // Cost model
  cost = estimate_cost(dag)

  // If cost too high, apply aggressive optimizations
  if cost.tokens > budget then
    dag = reduce_token_usage(dag)
    dag = enable_streaming(dag)

  return dag_to_workflow(dag)
}

// Example optimization:
original:
  research -> fetch_lib1 -> fetch_lib2 -> fetch_lib3 -> design

optimized:
  research -> (fetch_lib1 || fetch_lib2 || fetch_lib3) -> design

  Time: 20 -> 10 min (2x speedup)
  Tokens: same (parallel doesn't cost more)
```

Visual Model:

```
WORKFLOW OPTIMIZATION:

Input Workflow (suboptimal):
```


```
Time: 5 + 3 + 3 + 3 + 10 = 24 min

OPTIMIZATION PASSES:

Pass 1: Detect Parallelism
  Analyze dependencies:
    - fetch_l1, fetch_l2, fetch_l3 are independent
    - Can execute in parallel

Pass 2: Eliminate Redundancy
  No redundant steps found

Pass 3: Reorder for Locality
  Group independent fetches together

Pass 4: Insert Caching
```

```
  Add cache layer for library docs

Pass 5: Balance Load
  Distribute work evenly across parallel streams

Output Workflow (optimized):
```




```
Time: 5 + max(3,3,3) + 10 = 18 min
Speedup: 24/18 = 1.33x

Optimization Metrics:
```

Metric	Before	After
Time (min)	24	18
Tokens	45K	45K
Parallelism	0	3
Cache hits	0	80%
Steps	5	5

9.7 Level 6 Examples

Example 6.1: Generic Microservice Generator

```
// Meta-workflow that generates complete microservices
meta_workflow microservice_generator⟨Domain, Language⟩(
  domain_spec: DomainSpec,
  language: Language
) -> Project {

  // Type-level dispatch based on language
  type_check(Language in [Python, Rust, Go, TypeScript])

  // Generate domain model
  domain_model = generate_domain_model⟨Domain⟩(domain_spec)

  // Generate language-specific code
  code_generator = match language {
    Python     => python_generator(FastAPI, SQLAlchemy),
    Rust       => rust_generator(Actix, SeaORM),
    Go         => go_generator(Gin, GORM),
    TypeScript => ts_generator(Express, TypeORM)
  }

  // Generate project structure
  project = code_generator.generate(domain_model)

  // Generate tests (parallel for all domains)
  tests = map(domain_model.entities, |entity| {
    generate_tests⟨entity.type⟩(entity, language)
  })

  // Generate infrastructure
  infra = parallel [
    generate_docker⟨Language⟩(language),
    generate_k8s⟨Domain⟩(domain_spec),
    generate_ci_cd⟨Language⟩(language, tests)
  ]

  // Package everything
  return package_project(project, tests, infra)
}

// Instantiations
user_service_python = microservice_generator⟨UserManagement, Python⟩(
  user_domain_spec,
  Python
```

```
)

payment_service_rust = microservice_generator⟨Payments, Rust⟩(
  payment_domain_spec,
  Rust
)
```

Example 6.2: Self-Optimizing Research Pipeline

```
// Workflow that optimizes itself based on execution history
meta_workflow self_optimizing_research(
  topic: Topic,
  history: ExecutionHistory
) -> OptimizedWorkflow {

  // Analyze past performance
  performance_model = train_performance_model(history)

  // Predict optimal strategy
  strategy = performance_model.predict(topic)

  // Generate workflow based on predicted optimal strategy
  workflow = match strategy {
    DeepAnalysis => {
      (
        academic_research(topic) ||
        industry_research(topic) ||
        open_source_research(topic)
      ) ->
      synthesize_comprehensive ->
      verify_claims ->
      generate_report
    },

    QuickReference => {
      /ctx7(related_libraries) ->
      extract_key_points ->
      generate_summary
    },

    Comparative => {
      alternatives = detect_alternatives(topic)
      parallel(map(alternatives, research)) ->
      create_comparison_matrix ->
      provide_recommendations
    }
  }

  // Add instrumentation for future optimization
  return instrument_workflow(workflow, |metrics| {
    history.append(metrics)
    performance_model.update(metrics)
```

```
    })
}
```

Example 6.3: Monadic Error Recovery Workflow

```
// Sophisticated error handling with monadic composition
workflow resilient_deployment = do {
  -- Build phase
  build_result <- try_with_retry(3) {
    compile_code >>
    run_tests >>
    build_artifacts
  } catch BuildError as err -> {
    log_error(err) >>
    notify_developers(err) >>
    return Failure(err)
  }

  -- Deployment phase (only if build succeeded)
  deploy_result <- for env in [Staging, Production] {
    canary <- deploy_canary(env, build_result, percentage=5)

    metrics <- monitor(canary, duration=10.minutes)

    if metrics.error_rate < 0.01 then {
      full_deployment <- progressive_rollout(env, build_result)
      return Success(full_deployment)
    } else {
      rollback <- automatic_rollback(env, canary)
      analysis <- analyze_failure(metrics)

      -- Retry with fix if analysis suggests simple fix
      if analysis.fixable then {
        fix <- apply_auto_fix(analysis)
        retry deploy_canary(env, fix)
      else {
        alert_on_call(analysis) >>
        return Failure(analysis)
      }
    }
  }

  -- Success path
  return deploy_result
}
```

Example 6.4: Type-Safe Multi-Language Polyglot System

```
// Type-level guarantees for polyglot microservices
type family ServiceLanguage(service: ServiceName) where
  ServiceLanguage("gateway")      = Go
  ServiceLanguage("ml-inference") = Python
  ServiceLanguage("realtime")     = Rust
  ServiceLanguage("api")          = TypeScript

// Ensure correct toolchain is used per service
deploy_service(S: ServiceName)(service: S) -> Deployment {
  type Lang = ServiceLanguage(S)

  // Compiler ensures we use correct toolchain for language
  toolchain = match Lang {
    Go         => go_toolchain,
    Python     => python_toolchain,
    Rust       => rust_toolchain,
    TypeScript => node_toolchain
  }

  // Type-safe deployment
  build(service, toolchain) ->
  test(service, Lang::test_framework) ->
  containerize(service, Lang::base_image) ->
  deploy(service, Lang::runtime)
}

// Compile-time error if wrong service/language pair
deploy_service("ml-inference")  -- Uses Python toolchain ✓
deploy_service("realtime")      -- Uses Rust toolchain ✓
```

Part III: Advanced Topics

10. Optimization Patterns

10.1 Minimize Sequential Dependencies

Anti-pattern: Unnecessary sequential steps

```
❌ BAD:
analyze_frontend ->
analyze_backend ->
analyze_database ->
analyze_infrastructure
```

Optimized: Maximize parallelism

```
✅ GOOD:
analyze_frontend ||
analyze_backend ||
analyze_database ||
analyze_infrastructure
```

Impact:

```
Sequential: 4 × 15 min = 60 min
Parallel:   max(15, 15, 15, 15) = 15 min
Speedup:    4x
```

10.2 Batch Context Loading

Anti-pattern: Load context repeatedly

```
❌ BAD:
agent1 -> /ctx7("lib") -> agent2 -> /ctx7("lib") -> agent3
```

Optimized: Load once, share context

```
✅ GOOD:
ctx = /ctx7("lib")
agent1[ctx] -> agent2[ctx] -> agent3[ctx]
```

10.3 Early Failure Detection

Pattern: Validate early to fail fast

```
✅ GOOD:
validate_inputs ->
if(invalid) {
  return ValidationError
} else {
  expensive_operation ->
  more_expensive_operation
}
```

10.4 Resource Pooling

Pattern: Reuse expensive resources

```
✅ GOOD:
pool = create_agent_pool(size=10)

tasks = map(large_dataset, |item| {
  agent = pool.acquire()
  result = agent.process(item)
  pool.release(agent)
  return result
})
```

---

11. Error Handling Strategies

11.1 Retry with Exponential Backoff

```
Attempt 1: Immediate
Attempt 2: Wait 1s
Attempt 3: Wait 2s
Attempt 4: Wait 4s
Attempt 5: Wait 8s
...
Max wait: 60s
```

11.2 Circuit Breaker Pattern

```
States:
  CLOSED: Normal operation
    └─ too many failures ─▶ OPEN

  OPEN: Block all requests
    └─ timeout ─▶ HALF_OPEN

  HALF_OPEN: Allow test request
    ├─ success ─▶ CLOSED
    └─ failure ─▶ OPEN
```

11.3 Graceful Degradation

```
primary_service ->
catch(ServiceUnavailable) {
  cached_data || fallback_service || default_response
}
```

---

12. Resource Management

12.1 Token Budget Allocation

```
Total Budget: 100K tokens

Allocation:
  Research phase:    25K (25%)
  Design phase:      30K (30%)
  Implementation:    35K (35%)
  Testing phase:     10K (10%)

Reserve: 5K (5%) for overhead and merge operations
```

12.2 Time-Based Constraints

```
workflow with_timeout {
  result = timeout(30.minutes) {
    long_running_operation
  } catch TimeoutError {
    partial_results || cached_fallback
  }
}
```

12.3 Concurrency Limits

```
Max Concurrent: 5 agents
Queue: FIFO with priority

High priority: User-facing tasks
Normal priority: Background jobs
Low priority: Batch processing
```

---

13. Real-World Case Studies

13.1 E-Commerce Platform Migration

Scenario: Migrate monolith to microservices

```
workflow ecommerce_migration {
  // Phase 1: Analysis (parallel)
  (
    analyze_monolith ||
    identify_bounded_contexts ||
    map_dependencies
  ) ->

  // Phase 2: Strangler fig pattern
  for service in prioritized_services {
    extract_service(service) ->
    deploy_alongside_monolith(service) ->
    route_traffic_gradually(service, [10%, 50%, 100%]) ->
    retire_monolith_module(service)
  } ->

  // Phase 3: Data migration
  migrate_database_per_service ->
  setup_event_driven_communication ->

  // Phase 4: Validation
  run_integration_tests ->
  performance_regression_tests ->

  // Phase 5: Cutover
  final_cutover ->
  monitor_and_optimize
}
```

13.2 ML Model Training Pipeline

Scenario: Distributed model training and deployment

```
workflow ml_pipeline {
  // Data preparation (parallel)
  datasets = (
    fetch_training_data ||
    fetch_validation_data ||
    fetch_test_data
  )

  processed_data = map(datasets, |dataset| {
    clean_data(dataset) ->
    feature_engineering(dataset) ->
    normalize(dataset)
  })

  // Hyperparameter tuning (parallel)
  configs = generate_hyperparameter_configs(100)

  results = map(configs, |config| {
    train_model(processed_data, config) ->
    validate_model(processed_data.validation, config) ->
    return (model, metrics)
  })

  // Select best model
  best_model = select_top_k(results, k=3) ->
  ensemble_model = create_ensemble(best_model)

  // Deployment
  deploy_to_staging(ensemble_model) ->
  a_b_test(ensemble_model, current_production_model) ->
  if(ensemble_performs_better) {
    deploy_to_production(ensemble_model)
  }
}
```

---

14. Operator Reference

Complete Operator Table

| Operator | Type | Precedence | Associativity | Description | |----------|------|------------|---------------|-------------| | `()` | Grouping | 1 (highest) | N/A | Control precedence | | `[]` | Annotation | 2 | Left | Metadata/constraints | | `+` | Combination | 3 | Left | Merge capabilities | | `->` | Sequence | 4 | Right | Pipeline | | `\|\|` | Parallel | 5 | Left | Concurrent execution | | `:` | Specification | 6 | Right | Task binding | | `=` | Assignment | 7 (lowest) | Right | Parameter binding | ---

15. Pattern Library

Core Patterns

1. **Pipeline**: `A -> B -> C` 2. **Fan-out/Fan-in**: `A -> (B \|\| C \|\| D) -> E` 3. **Map-Reduce**: `map(items, f) -> reduce(results, merge)` 4. **Conditional**: `if(p) A : B` 5. **Retry**: `retry(n, backoff) A : fallback` 6. **Context Sharing**: `agent1[ctx] -> agent2[ctx]` ---

16. Mathematical Laws

Algebraic Laws

```
Combination (+):
  A + B = B + A                    (commutative)
  (A + B) + C = A + (B + C)        (associative)
  A + ∅ = A                        (identity)

Sequence (->):
  (A -> B) -> C = A -> (B -> C)     (associative)
  A -> B ≠ B -> A                  (not commutative)
  id -> A = A -> id = A            (identity)

Parallel (||):
  A || B = B || A                  (commutative)
  (A || B) || C = A || (B || C)    (associative)
  A || ∅ = A                       (identity)

Distribution:
  A -> (B || C) = (A -> B) || (A -> C)
  (A || B) -> C = (A -> C) || (B -> C)
```

---

17. Appendices

A. Glossary

- **Agent**: Autonomous worker with specialized capabilities - **Skill**: Knowledge module that augments agents - **Workflow**: Composed orchestration with execution semantics - **DAG**: Directed Acyclic Graph representing dependencies - **Fork/Join**: Parallel execution pattern - **Functor**: Structure-preserving map operation - **Monad**: Abstraction for sequential composition with context

B. Further Reading

- Category Theory for Programmers - Functional Programming in Scala - Design Patterns (Gang of Four) - Domain-Specific Languages (Martin Fowler)

C. Complexity Summary

| Level | Time (min) | Tokens (K) | Use Case | |-------|----------|----------|---------| | 1 | 2-5 | 5-15 | Quick tasks | | 2 | 5-15 | 10-30 | Simple pipelines | | 3 | 20-45 | 40-100 | Multi-stream research | | 4 | 45-90 | 80-150 | Complex orchestration | | 5 | 90-180 | 120-250 | Workflow libraries | | 6 | 180+ | 200+ | Meta-programming | --- **End of Comprehensive DSL Orchestration Reference** **Version**: 2.0.0 **Date**: 2025-10-19 **Total Pages**: ~150 (formatted) **Total Examples**: 50+ **Total Diagrams**: 80+ ASCII visualizations ---